Indice

1. Nozioni preliminari ... 4
   1. 1. Funzioni one-way e funzioni trapdoor ... 4
2. Crittografia simmetrica ... 5
   2. 1. Cifrario di Cesare ... 5
   2. 2. Crittoanalisi per frequenze ... 5
   2. 3. Cifrario di Vigenère ... 5
   2. 4. One-time pad ... 5
   2. 5. Obiettivi di sicurezza di un cifrario ... 6
   2. 6. Cifrari a blocchi ... 6
      2. 6. 1. Modelli d'attacco ... 7
   2. 7. Feistel network ... 7
      2. 7. 1. Funzione F ... 8
   2. 8. Mode of operation ... 8
      2. 8. 1. ECB mode ... 8
      2. 8. 2. CBC mode ... 9
3. Aritmetica modulare ... 10
   3. 1. Insiemi $\mathbb{Z}_n$ ... 10
   3. 2. Vantaggio computazionale dell'aritmetica modulare ... 10
   3. 3. Algoritmo di Euclide ... 10
      3. 3. 1. Efficienza dell'algoritmo ... 11
   3. 4. Teorema cinese dei resti ... 11
      3. 4. 1. Dimostrazione ... 11
         3. 4. 1. 1. Corrispondenza $\mathbb{Z}_n \Longrightarrow \mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \times ... \times \mathbb{Z}_{n_r}$ ... 11
         3. 4. 1. 2. Corrispondenza $\mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \times ... \times \mathbb{Z}_{n_r} \Longrightarrow \mathbb{Z}_n$ ... 11
   3. 5. Gruppi e gruppi ciclici ... 12
   3. 6. Safe prime ... 13
   3. 7. Esponenziale modulare ... 13
      3. 7. 1. Logaritmo discreto ... 13
         3. 7. 1. 1. Algoritmo baby-steps giant-steps ... 14
4. Diffie-Hellman key-exchange protocol ... 15
   4. 1. Efficienza ... 15
   4. 2. Sicurezza ... 15
   4. 3. Nota sul segreto condiviso g^{ab} ... 15
   4. 4. Problema dell'autenticazione ... 16
   4. 5. Chiavi DH effimere ... 16
5. Casualità e algoritmi probabilistici ... 17
   5. 1. Algoritmi probabilistici ... 17
      5. 1. 1. Algoritmi di tipo Monte Carlo ... 17
         5. 1. 1. 1. Esempio di algoritmo NON è probability-bounded ... 17
         5. 1. 1. 2. Importanza del limite sull'errore probabilistico ... 18
      5. 1. 2. Algoritmi di tipo Las Vegas ... 18
         5. 1. 2. 1. Esempio: algoritmo per aggiustare l'uniformità di una sequenza casuale ... 18
   5. 2. Numeri primi e test di primalità ... 19
      5. 2. 1. Test di primalità basato sul piccolo teorema di Fermat ... 19
      5. 2. 2. Test di primalità di Solovay-Strassen ... 20
   5. 3. Fattorizzazione di interi ... 22
      5. 3. 1. Algoritmo di fattorizzazione ρ di Pollard ... 22
         5. 3. 1. 1. Casi sfavorevoli ... 23
6. RSA ... 24

6. 1. Dimostrazione della correttezza ... 24
6. 2. Efficienza di RSA ... 25
6. 3. Sicurezza di RSA ... 25
6. 3. 1. Malleabilità ... 25
6. 3. 2. e troppo piccolo ... 25
6. 3. 3. Algoritmo di fattorizzazione di Fermat ... 25
6. 3. 3. 1. Scelta del punto di partenza ... 26
6. 3. 3. 2. Efficienza dell'algoritmo ... 26
6. 3. 4. Riutilizzo di p e q ... 27
6. 4. Aspetti implementativi di RSA ... 27
6. 4. 1. Velocizzare la decifratura ... 27
6. 5. Optimal Asymmetric Encryption Padding ... 28
7. Protocollo di Rabin ... 30
7. 1. Calcolo delle radici quadrate modulari ... 30
7. 2. Cifratura e decifratura ... 30
7. 3. Dimostrazione dell'equivalenza alla fattorizzazione ... 31
8. Protocollo di ElGamal ... 32
8. 1. Riuso di b ... 32
8. 2. Sicurezza - ElGamal vs RSA ... 32
9. Firma digitale ... 33
9. 1. Uso delle funzioni hash durante la firma digitale ... 33
9. 2. Firma digitale con RSA ... 33
9. 2. 1. Blinding attack ... 33
9. 3. Firma digitale con ElGamal ... 34
9. 3. 1. Verifica della firma ... 34
9. 3. 2. Riuso di k ... 35
9. 3. 3. Message forgery attack ... 35
9. 4. Firma digitale con DSA ... 35
9. 4. 1. Generazione delle chiavi ... 35
9. 4. 2. Firma ... 36
9. 4. 3. Verifica della firma ... 36
9. 4. 3. 1. Dimostrazione della correttezza ... 36
9. 4. 4. Generazione di p ... 37
9. 4. 5. Ottenere g ... 37
9. 4. 6. Sicurezza ... 37
9. 5. Autenticità delle chiavi pubbliche ... 37
9. 5. 1. Approccio TLS ... 37
9. 5. 2. Approccio OpenPGP/GPG ... 38
10. Crittografia su curve ellittiche ... 39
10. 1. Campi finiti ... 39
10. 2. Definizione di curva ellittica ... 39
10. 3. Curve ellittiche smooth ... 40
10. 4. Intersezioni con una retta ... 40
10. 5. Punto all'infinito ... 41
10. 6. Addizione ... 41
10. 6. 1. Calcolo delle coordinate di C ... 41
10. 6. 1. 1. Caso $A \neq B$ ... 41
10. 6. 1. 2. Caso $A = B$ (retta verticale) ... 42
10. 7. Curve ellittiche su $\mathbb{Z}_p$ ... 43
10. 7. 1. Logaritmo discreto su curva ellittica ... 43
10. 7. 2. Numero di punti su una curva ... 43
10. 7. 3. Attacco al logaritmo discreto su curve ellittiche ... 44

10. 7. 3. 1. Complessità dell'algoritmo 44
10. 8. Diffie-Hellman su curve ellittiche (ECDH) 44
10. 8. 1. Attacco basato sull'uso di una curva debole 45
10. 8. 1. 1. Esempio 45
10. 8. 1. 2. Considerazioni 46
10. 9. DSA su curve ellittiche (ECDSA) 46
10. 9. 1. Dimostrazione della correttezza 47
10. 9. 2. Attacco in caso di riuso di k 47
10. 10. Vantaggi e svantaggi delle curve ellittiche 47

1. Nozioni preliminari

confidenzialità	il messaggio dev'essere comprensibile solo alle persone autorizzate
autenticazione del mittente	il destinatario dev'essere certo dell'**identità** di chi gli ha mandato il messaggio
integrità	il destinatario deve avere modo di accorgersi se il messaggio è stato alterato in un qualche modo rispetto a quello inviato dal mittente
non ripudio	il mittente non può negare di aver inviato il messaggio ed il destinatario non può negare di averlo ricevuto

Tabella 1: Requisiti per una comunicazione sicura

Principio di Kerckhoff: in uno schema di cifratura, la sicurezza deve risiedere **solo** nella **segretezza della chiave**. Non deve risiedere nella segretezza dell'algoritmo.

	Crittografia simmetrica	**Crittografia asimmetrica**
chiavi per partecipante	una sola chiave	due chiavi distinte, una per cifrare e l'altra per decifrare
problema principale	mantenere segreta la chiave	**autenticare** le parti coinvolte nella comunicazione
scalabilità	limitata perché n partecipanti occorrono $O(n^2)$ chiavi totali (ogni partecipante deve avere $n-1$ chiavi)	buona perché il numero di chiavi è $O(2n)$ (ogni partecipante ha una sola coppia di chiavi)
costo computazionale	basso grazie ad algoritmi estremamente efficienti (anche con supporto hardware)	alto a causa di algoritmi lenti basati su **problemi matematici**

Tabella 2: Differenze tra crittografia simmetrica e asimmetrica

1. 1. Funzioni one-way e funzioni trapdoor

Se f è una **funzione one-way**, significa che è facile calcolare $f(x)$, ma è **computazionalemnte complesso** (o impossibile) calcolare $f^{-1}(x)$.

Se f è una **funzione trapdoor**, anche calcolare $f^{-1}(x)$ è facile, ma **solo a determinate condizioni**. Nel contesto della crittografia asimmetrica, la condizione è **conoscere la chiave privata**.

La crittografia asimmetrica utilizza delle funzioni trapdoor e non delle funzioni one-way. Se la cifratura fosse fatta con una funzione one-way, **nemmeno il legittimo destinatario** riuscirebbe a rimettere in chiaro il messaggio cifrato, perché la funzione usata per la cifratura non è invertibile.

2. Crittografia simmetrica

Componenti di un cifrario simmetrico:

- **algoritmo**: applica una certa **permutazione**[1] ad una porzione del plaintext
- **mode of operation**: definisce come rimettere insieme i blocchi del ciphertext

2. 1. Cifrario di Cesare

Semplice cifrario mono-alfabetico in cui la permutazione consiste in uno **shift** della lettera di k posizioni in avanti:

$$c_i = p_i \oplus k$$

dove c_i e p_i sono rispettivamente l'i-esimo carattere cifrato e in chiaro.

La chiave segreta è proprio il valore k.

Il numero di possibili chiavi dipende dalla cardinalità dell'alfabeto considerato.

2. 2. Crittoanalisi per frequenze

Tutti i cifrari mono-alfabetici sono vulnerabili a crittoanalisi per frequenze, perché (in quanto mono-alfabetici) non sono in grado di mascherare l'**identità** di una lettera all'interno di un testo.

2. 3. Cifrario di Vigenère

Si tratta di un cifrario poli-alfabetico che, in un'epoca in cui non esistevano i computer, era considerato il cifrario più robusto (ma era anche complesso utilizzarlo concretamente).

Il cifrario esegue sempre un'operazione di **shifting**, ma il valore non è più uguale per tutti i caratteri, ma è determinato dal carattere della chiave:

$$c_i = p_i \oplus k_i$$

dove k_i è l'i-esimo carattere della chiave.

Se k è la lunghezza della chiave, le possibili permutazioni per ogni carattere del plaintext sono 26^k. Si tratta quindi di un cifrario molto robusto rispetto agli attacchi di forza bruta.

Se la chiave è relativamente corta rispetto al testo, si può però fare un attacco per individuarne la **lunghezza della chiave**:

1. si individuano due porzioni di testo uguali all'interno del ciphertext e si calcola la distanza (in numero di caratteri) tra di loro;
2. è molto probabile che la lunghezza della chiave sia pari a questa distanza oppure ad un suo **divisore**;
3. si può provare una crittoanalisi per frequenze (o un **dictionary attack**) con le diverse ipotesi di lunghezza della chiave

2. 4. One-time pad

Si tratta dell'unico cifrario simmetrico per cui esiste una **dimostrazione matematica** del fatto che sia inviolabile (a determinate condizioni).

Nella pratica questo cifrario viene utilizzato raramente, perché richiede un'enorme quantità di **bit casuali**: la lunghezza della chiave infatti dev'essere pari alla lunghezza del messaggio.

Messaggio e chiave vengono interpretati come **sequenze di bit**. La cifratura consiste in uno XOR dei bit del messaggio con i bit della chiave:

$$c_i = p_i \oplus k_i$$

[1] dal punto di vista matematico, una permutazione è una funzione $f : I \to I$ invertibile

Per essere sicuro, la chiave dev'essere una sequenza **casuale** di bit, cioè ogni bit della chiave deve avere probabilità $= \frac{1}{2}$ di essere 0 o 1.

La chiave non deve mai essere riutilizzata per cifrare due messaggi distinti. Se ciò viene fatto, grazie alle proprietà dello XOR è possibile mettere in evidenza lo XOR dei due plaintext:

$$\begin{aligned} C_1 \oplus C_2 &= (P_1 \oplus K) \oplus (P_2 \oplus K) \\ &= P_1 \oplus P_2 \end{aligned}$$

Conoscendo uno fra P_1 e P_2, o una parte di uno/entrambi, invertendo le formule è possibile sia mettere in chiaro l'altro che risalire alla chiave segreta.

2. 5. Obiettivi di sicurezza di un cifrario

- **indistinguibilità**: per un eventuale eavesdropper, un messaggio cifrato dev'essere indistinguibile da una sequenza di bit casuali;
- non **malleabilità**: un cifrario si dice malleabile se dato un ciphertext C_1 è possibile crearne un altro il cui plaintext P_2 abbia una qualche **relazione forte** con P_1

2. 6. Cifrari a blocchi

In un cifrario a blocchi il plaintext viene diviso in varie parti (**blocchi**), ognuna con una dimensione fissa B. La **mode of operation** stabilisce come questi blocchi vengono rimessi insieme dopo che ogni blocco è stato cifrato.

La scelta sulla dimensione dei blocchi dev'essere "bilanciata":

- non può essere troppo grande, altrimenti si avrebbe molto overhead in caso di messaggi molto corti;
- non può essere troppo piccola, altrimenti si avrebbero delle falle di sicurezza. Se B è sufficientemente piccolo, l'attaccante può costruirsi una **lookup table** che mappa tutti i possibili plaintext con tutti i possibili ciphertext, rendendo molto semplice il processo di decifratura

I block cipher possono essere descritti con uno schema matematico astratto:

- se k è la lunghezza della chiave, ci sono 2^k possibili **tabelle di permutazione**. La chiave determina la tabella da utilizzare;
- se B è la dimensione del blocco, ogni tabella ha 2^B righe. Una volta stabilita la tabella da utilizzare, il processo di cifratura/decifratura si riduce ad un lookup a questa tabella

In realtà, poiché ognuna delle 2^k tabelle è generabile permutandone qualcun'altra, il numero di tabelle a disposizione è $(2^B)!$, perché questo è il numero di possibili permutazioni di B bit. Per poter utilizzare tutte queste permutazioni, chiave dev'essere di dimensioni molto grandi perché altrimenti non si potrebbero indicizzare tutte queste tabelle.

La dimensione della chiave per poter utilizzare $(2^B)!$ tabelle cresce vertiginosamente al crescere di B (già per $B = 7$ servirebbe una chiave da 621 bit per poter indicizzare tutte le $(2^B)!$ tabelle), rendendo di fatto **inutilizzabili** la stragrande maggioranza di queste $(2^B)!$ permutazioni.

Di questa piccola parte delle $(2^B)!$ permutazioni utilizzabili, ce ne sono delle altre ancora che non possono essere utilizzate perché aprirebbero delle falle di sicurezza (ad esempio non possono essere utilizzate tutte quelle permutazioni che lasciano in chiaro un qualche bit del plaintext). In generale, tutte le permutazioni che possono essere espresse come **trasformazioni lineari affini invertibili** sul campo $\mathbb{Z}_2$ non sono utilizzabili:

$$xP + b = y(x)$$

dove P è una matrice invertibile di dimensioni $B \times B$ su $\mathbb{Z}_2$ mentre x ed $y(x)$ sono due vettori riga di B elementi con coefficienti in $\mathbb{Z}_2$.

Se la trasformazione è di questo tipo, nel modello **chosen plaintext attack** l'attaccante può ricostruire b e P e dunque riuscire ad **invertire** la trasformazione:

- per determinare b è sufficiente chiedere all'oracolo di cifrare un blocco contenente solo degli zeri (1 query necessaria);
- per determinare P si effettuano B query all'oracolo di cifratura utilizzando ogni volta un **vettore della base canonica** diverso

Quindi con $B + 1$ query l'attaccante riesce a rompere la cifratura.

La trasformazione utilizzata per la cifratura deve quindi essere **non lineare**. Oltre a questa proprietà, le permutazioni che si possono utilizzare devono averne altre 2:

- **diffusione**: una piccola modifica nel plaintext deve avere un fortissimo **effetto valanga** su tutto il ciphertext;
- **confusione**: il ciphertext deve distruggere qualsiasi possibile **pattern** presente nel plaintext (ad esempio lettere ripetute)

2. 6. 1. Modelli d'attacco

In ordine decrescente di "potenza" da parte dell'attaccante:

1. **ciphertext-only attacker**: l'unica informazione nota all'attaccante è il ciphertext;
2. **known-plaintext attacker**: l'attaccante conosce una o più mappature plaintext $\rightarrow$ ciphertext, ma in generale non sa come decifrare un generico ciphertext;
3. **chosen plaintext attacker**: l'attaccante ha modo di fare delle **query di cifratura** ad un oracolo. Nella crittografia asimmetrica si ha **sempre** a che fare con questo tipo di attaccanti, perché la chiave è pubblica;
4. **chosen ciphertext attacker**: l'attaccante ha modo di fare delle **query di decifratura**, cioè è in grado di decifrare un qualunque ciphertext

2. 7. Feistel network

Non è un cifrario a blocchi di per sè, ma è uno **schema** per costruirli. Tra gli altri, è stata utilizzata per costruire il cifrario DES.

Una Feistel network è composta da un certo numero di **stadi** (o round), ognuno fatto sempre allo stesso modo. Il numero di stadi è strettamente legato al concetto di **diffusione**.

Ognuno di questi stadi lavora con 3 parametri:

- L_i ed R_i, che sono rispettivamente la parte sinistra e destra del ciphertext intermedio su lo stadio sta lavorando (nel caso del 1° stadio, L_0 è la parte sinistra del plaintext ed R_0 è la parte destra);
- k_i, che è una **round key** che viene **derivata** a partire dalla chiave generale k usata dall'algoritmo tramite un processo denominato **key schedule**

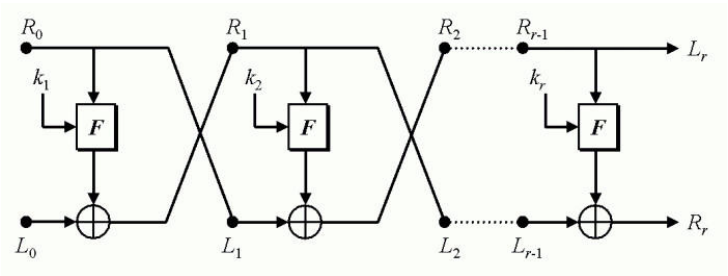

Figura 1: Feistel network

Come si vede dalla figura:

$$L_{i+1} = R_i \qquad R_{i+1} = F(k_i, R_i) \oplus L_i$$

dove F è una trasformazione che dipende dalla chiave k_i.

Invertire il processo di cifratura è molto semplice grazie alle proprietà dello XOR: basta invertire le frecce nello schema (cioè si parte dal fondo anziché dall'inizio). In realtà però si può fare ancora meglio: si parte sempre dall'inizio ma si invertono i ruoli di R_0 ed L_0, cioè R_0 diventa la parte **sinistra** del ciphertext ed L_0 diventa la parte **destra**.

2. 7. 1. Funzione F

La funzione F è composta da due blocchi:
- S-box: effettua una **sostituzione** dei bit;
- P-box: effettua una **permutazione** dei bit

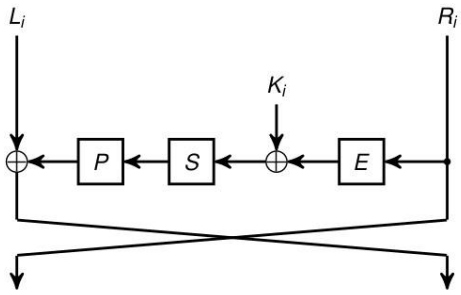

Figura 2: S-box e P-box

Lo scopo di F è duplice:
1. sfruttare la chiave nel processo, per mescolarla con i bit del messaggio;
2. forzare la **non linearità** della trasformazione da plaintext a ciphertext

Il modo in cui è implementata F dipende dal cifrario. Nel caso di DES:
1. si espandono i 32 bit di R_i in 48 bit, tramite **duplicazione** di alcuni bit;
2. si XORano questi 48 bit con quelli della round key k_i;
3. si riporta questo risultato ad un valore a 32 bit (per poterlo successivamente XORare con L_i) tramite l'S-box, che esegue un lookup ad una tabella:
   1. si divide il blocco da 48 bit in 8 blocchi da 6 bit ciascuno;
   2. i primi 2 bit di ogni blocco definiscono la riga da utilizzare per il lookup, mentre gli altri 4 definiscono la colonna;
   3. il risultato del lookup è un numero a 4 bit;
   4. si concatenano gli 8 risultati da 4 bit ottenuti (uno per ogni blocco da 6 bit), ottenendo un valore a 32 bit
4. si permuta il risultato ottenuto e poi lo si XORa con L_i

2. 8. Mode of operation

2. 8. 1. ECB mode

Si tratta dell'algoritmo più semplice (ma anche il più vulnerabile) con cui vengono rimessi insieme i blocchi dopo che sono stati cifrati.

1. si cifra ogni blocco, in maniera **indipendente** dagli altri;
2. il ciphertext finale è dato dalla concatenazione dei ciphertext di ciascun blocco

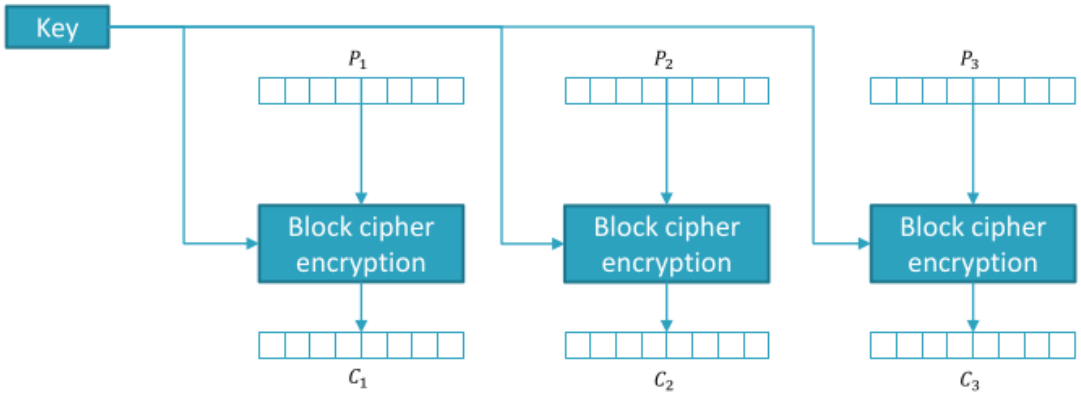

Figura 3: ECB mode

La vulnerabilità di questo approccio sta proprio nel fatto che i blocchi sono cifrati in modo **indipendente**: lo stesso blocco di plaintext viene cifrato sempre allo stesso modo, rendendo poco robusta la cifratura.

2. 8. 2. CBC mode

Prima di procedere con la cifratura dell'i-esimo blocco di plaintext:

- per $i = 1$ si XORa il blocco con l'**initialization vector**
- per $i > 1$ lo XOR viene fatto con il blocco di ciphertext precedente

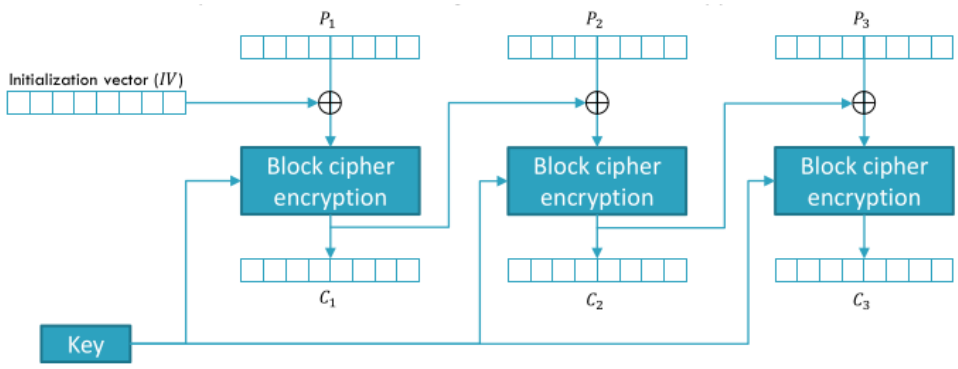

Figura 4: CBC mode

L'IV deve essere **sempre diverso** ed è fondamentale che sia scelto in modo **random**.

L'IV non è un'informazione segreta: viene inviato assieme al ciphertext su un canale pubblicamente accessibile (potrebbe quindi essere intercettato).

3. Aritmetica modulare

3. 1. Insiemi $\mathbb{Z}_n$

Insiemi di numeri **interi** che vanno da 0 ad $n-1$:

$$\mathbb{Z}_n = \{0, 1, ..., n-1\}$$

L'insieme $\mathbb{Z}_n$ può essere visto anche come l'**insieme dei possibili resti** che si ottengono dividendo gli $n-1$ numeri per n:

$$\mathbb{Z}_n = \{i \bmod n\} \qquad \forall i = 0, 1, ..., n-1$$

Negli insiemi $\mathbb{Z}_n$ sono sempre definite le operazioni di **somma**, **sottrazione** e **moltiplicazione**:

$$\begin{aligned} x +_n y &= (x+y) \bmod n \\ x -_n y &= (x-y) \bmod n \\ x \cdot_n y &= (xy) \bmod n \end{aligned}$$

La sottrazione è definita anche quando $x - y < 0$, perché il resto della divisione intera per n è comunque un valore in $\mathbb{Z}_n$ (es. $(5-8) \bmod 7 = 4$).

L'**opposto** di ogni elemento è definito: si tratta del valore y tale che $(x+y) \bmod n = 0$, da cui risulta $y = n - x$.

L'**inverso moltiplicativo** invece esiste solo per gli elementi $x \in \mathbb{Z}_p$ che sono **coprimi** con n. Segue quindi che se n è primo, ogni elemento di $\mathbb{Z}_n \setminus \{0\}$ ha inverso moltiplicativo.

Proprietà del modulo rispetto ai suoi divisori: se m è un divisore di n, allora per ogni $x \in \mathbb{Z}$ vale:

$$(x \bmod n) \bmod m = x \bmod m$$

3. 2. Vantaggio computazionale dell'aritmetica modulare

Lavorare con l'aritmetica modulare è un grosso vantaggio perché permette di **mantenere piccoli i numeri**. Quando si esegue un'operazione in modulo, si può sceglie se:

- applicare il modulo **solo alla fine** (es. $(x+y+z) \bmod n$);
- applicare il modulo **ad ogni passo** (es. $[(x \bmod n) + (y \bmod n) + (z \bmod n)] \bmod n$)

3. 3. Algoritmo di Euclide

Algoritmo estremamente efficiente per il calcolo del **massimo comune divisore** tra due numeri. La sua efficienza è dovuta all'uso dell'aritmetica modulare.

$$\gcd(x, y) = \begin{cases} x \text{ se } y = 0 \\ \gcd(y, x \bmod y) \text{ altrimenti} \end{cases}$$

Esiste anche una versione estesa dell'algoritmo:

$$\text{ExtEuclid}(x, y) = \begin{cases} (x, 1, 0) \text{ if } y = 0 \\ \left(m, b, a - b\left\lfloor \frac{x}{y} \right\rfloor\right) \text{ if } (m, a, b) = \text{ExtEuclid}(y, x \bmod y) \end{cases}$$

dove $m = \gcd(x, y)$, mentre a e b sono due numeri che soddisfano la **Bezout's identity**:

$$\gcd(x, y) = m = ax + by$$

Se $\gcd(x, n) = 1$, la versione estesa dell'algoritmo può essere utilizzata per **calcolare l'inverso modulare** di $x \in \mathbb{Z}_n$:
1. si parte dalla Bezout's identity: $m = ax + by$;
2. se $m = 1$ (unico caso in cui può esistere l'inverso), si riscrive l'espressione come $ax = 1 - bn$;
3. si applica il modulo n ad entrambi i membri, ottenendo $(ax) \bmod n = 1$
4. l'inverso di x è dunque a

3. 3. 1. Efficienza dell'algoritmo

L'algoritmo di Euclide (in entrambe le sue versioni) ha un **costo lineare** nella dimensione in bit dei parametri. Grazie all'utilizzo del modulo, ogni **2 iterazioni** la dimensione dei parametri si **dimezza**.

3. 4. Teorema cinese dei resti

Il teorema cinese dei resti è importante sia dal punto di vista teorico che dal punto di vista pratico, in quanto permette di velocizzare alcune operazioni (ad esempio il processo di decifratura in RSA).

Sia n un intero esprimibile come **prodotto** di $r > 1$ interi tutti **relativamente primi** tra loro:

$$n = n_1 \cdot n_2 \cdot ... \cdot n_r \qquad \gcd\left(n_i, n_{j \neq i}\right) = 1$$

Il resto della divisione di un qualsiasi intero a per n è completamente determinato dai resti delle divisioni per $n_1, n_2, ..., n_r$. In altre parole, esiste una **corrispondenza biunivoca** tra l'insieme $\mathbb{Z}_n$ e l'insieme dato dal **prodotto cartesiano** $\mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \times ... \times \mathbb{Z}_{n_r}$.

3. 4. 1. Dimostrazione

3. 4. 1. 1. Corrispondenza $\mathbb{Z}_n \Longrightarrow \mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \times ... \times \mathbb{Z}_{n_r}$

Dato un valore $a \in \mathbb{Z}_n$, definire la tupla degli r-possibili resti è immediato:

$$a \longrightarrow (a \bmod n_1, a \bmod n_2, ..., a \bmod n_r)$$

Ogni elemento $a \bmod n_i$ della tupla è un valore in $\mathbb{Z}_{n_i}$.

3. 4. 1. 2. Corrispondenza $\mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \times ... \times \mathbb{Z}_{n_r} \Longrightarrow \mathbb{Z}_n$

Per dimostrare questa corrispondenza, bisogna trovare i coefficienti $c_i \in \{0, 1\}$ che soddisfano la combinazione lineare

$$C = c_1 a_1 + c_2 a_2 + ... + c_r a_r$$

tali che $C \bmod n_i = a_i$ per $i = 1, ..., r$, con $a_i \in \mathbb{Z}_{n_i}$.

Se si riescono a trovare questi coefficienti, allora, data la **cardinalità** dei due insiemi considerati ($\mathbb{Z}_n$ e l'insieme prodotto cartesiano), significa che $C = a$ (con $a \in \mathbb{Z}_n$).

Per trovare i c_i:
1. per ogni $i = 1, ..., r$, si calcola m_i come il prodotto di tutti i moduli $n_1, ..., n_r$ ad eccezione dell'i-esimo:

$$m_i = \prod_{j \neq i} n_j$$

2. dato che m_i non contiene n_i tra i suoi coefficienti, e siccome tutti gli n_i sono primi tra loro, $\gcd(m_i, n_i) = 1$ e dunque esiste l'**inverso** di m_i modulo n_i;
3. si definisce $c_i = m_i \cdot (m_i^{-1} \bmod n_i)$

A questo punto possono succedere 2 cose:

- per $j \neq i$, $c_i \bmod n_j = 0$ perché $c_i = m_i \cdot (m_i^{-1} \bmod n_i)$ è un multiplo di n_j, dato che m_i contiene n_j al suo interno come fattore;
- per $i = j$, $c_i \bmod n_i = 1$ grazie alle proprietà del modulo:

$$\begin{aligned} c_i \bmod n_i &= [m_i \cdot (m_i^{-1} \bmod n_i)] \bmod n_i \\ &= (m \cdot m_i^{-1}) \bmod n_i \\ &= 1 \end{aligned}$$

A questo punto, se $C = \sum_{i=1}^{r} c_i a_i$ allora $C \bmod n_i = a_i$ per $i = 1, ..., r$, dunque C ha gli stessi resti di a per ciascuno degli n_i.

Dato che la cardinalità dei due insiemi è la stessa, deve necessariamente risultare che $C = a$, altrimenti significherebbe che esiste un elemento di $\mathbb{Z}_n$ a cui non corrisponde nessun elemento dell'insieme prodotto cartesiano (cosa impossibile, perché i due insiemi hanno la stessa cardinalità).

3. 5. Gruppi e gruppi ciclici

Un gruppo è un insieme numerico dov'è definita un'operazione che soddisfa 4 proprietà:
1. **chiusura** rispetto al gruppo;
2. **associatività**
3. esistenza dell'**elemento neutro**
4. esistenza dell'**inverso**

Gli insiemi $\mathbb{Z}_n$ sono dunque dei gruppi rispetto alle operazioni di somma (gruppo **additivo** $\mathbb{Z}_n^+$).

Solo nel caso in cui n è primo, l'insieme $\mathbb{Z}_n$ è un gruppo anche rispetto all'operazione di moltiplicazione (gruppo **moltiplicativo** $\mathbb{Z}_n^*$). Se n non è primo non si può parlare di gruppo rispetto alla moltiplicazione, perché non tutti gli elementi di $\mathbb{Z}_n$ hanno inverso modulare.

I gruppi moltiplicativi sono dei **campi finiti**, in quanto vale anche la proprietà distributiva della moltiplicazione rispetto all'addizione.

L'**ordine** di un elemento x di un generico gruppo G è il numero di volte con cui si può sommare (o moltiplicare, per i gruppi moltiplicativi) x a sè stesso (partendo dall'elemento neutro) prima di riottenere ~~nuovamente x come risultato~~. l'elemento neutro.

L'ordine di un generico gruppo G invece è il **numero di elementi** contenuti nel gruppo stesso.

Un gruppo si dice **ciclico** se esiste **almeno un elemento** il cui ordine è pari a quello del gruppo. Tale elemento si dice **generatore** (o **radice primitiva**). Se n è primo, $\mathbb{Z}_n$ è sempre un gruppo ciclico (sia nel caso additivo che nel caso moltiplicativo).

Se g è un generatore e p è un numero primo, tutte le potenze intere di g fino a $p-1$ formano un gruppo moltiplicativo:

$$\mathbb{Z}_p^* = \{g^i \bmod p \mid i = 1, ..., p-1\}$$

Questa cosa vale anche per ogni elemento $h \in \mathbb{Z}_p^*$ di ordine $s < p-1$. h non è un generatore del gruppo, ma le sue potenze fino ad s generano un **sottogruppo ciclico**:

$$H = \{h^i \bmod p \mid i = 1, ..., s\}$$

Teorema di Lagrange: tutti i sottogruppi di un gruppo di ordine k hanno ordine pari ad un **divisore** di k.

Teorema fondamentale dei gruppi ciclici: per ogni divisore k dell'ordine del gruppo esiste **uno ed un solo** sottogruppo di ordine k

3. 6. Safe prime

Un **safe prime** è un numero primo p del tipo $p = 2q + 1$, con q primo a sua volta.

L'ordine di $\mathbb{Z}_p$ è $p - 1 = 2q$, dunque $\mathbb{Z}_p$ ha due soli sottogruppi:
- quello banale, di ordine 2, $\{1, -1\}$;
- il sottogruppo di ordine q

Escludendo il sottogruppo banale $\{1, -1\}$, la metà degli elementi di $\mathbb{Z}_p$ genera $\mathbb{Z}_p$ stesso, mentre l'altra metà genera il sottogruppo di ordine q. Dunque un qualunque elemento x può avere o ordine q o ordine $p - 1$.

Il teorema di Lagrange, preso un valore $x \in \mathbb{Z}_p$, permette di calcolare immediatamente l'ordine di x: se $x^q \bmod p = 1$ allora x ha ordine q, altrimenti ha ordine $p - 1$.

Il sottogruppo di ordine q è formato dai **residui quadratici** modulo p, ovvero da quei numeri y tali che $x^2 \bmod p = y$. Come nel caso reale, poiché p è primo, y in realtà ha **due** radici, che sono l'una l'opposto dell'altra. Entrambe queste radici fanno parte del sottogruppo di ordine q.

3. 7. Esponenziale modulare

L'aritmetica modulare permette di calcolare efficientemente il valore di un'esponenziale anche quando si ha a che fare con numeri molto grandi.

L'idea dell'algoritmo per l'esponenziale modulare parte dal calcolo del prodotto modulare. Un'espressione del tipo $z = (ab) \bmod n$ può essere calcolata efficientemente in questo modo:
1. si inizializza un accumulatore $z = 0$;
2. si scrive b in forma binaria e si itera su ogni suo bit;
3. se l'i-esimo bit di b ha valore 1, si aggiunge all'accumulatore il valore $a_k = a \cdot 2^i$;

```
def mod_prod(a: int, b: int, n: int) -> int:
    a %= n
    b %= n
    z = 0
    while b != 0:
        if b & 1:
            z = (z + a) % n
        a = (a << 1) % n
        b >>= 1
    return z
```

Codice 1: Algoritmo per il calcolo del prodotto modulare

L'algoritmo per l'esponenziale modulare sfrutta lo stesso trucco, ma l'accumulazione viene fatta per moltiplicazione (anziché per addizione):

```
def mod_exp(a: int, b: int, n: int) -> int:
    a %= n
    b %= n
    z = 1
    while b != 0:
        if b & 1:
            z = (z * a) % n
        a = (a * a) % n
        b >>= 1
    return z
```

Codice 2: Algoritmo per il calcolo dell'esponenziale modulare

3. 7. 1. Logaritmo discreto

Se $b^e = x \bmod n$, l'esponente e è detto **logaritmo discreto** in base b di x e si indica come $e = \log_{b(x)} \bmod n$. Il logaritmo discreto è definito anche sui gruppi additivi: il logaritmo discreto in base g di x è il valore k tale per cui $k \cdot g = x \bmod n$.

Il calcolo del logaritmo discreto, anche conoscendo la base b ed il modulo n, è un **problema difficile**.

Il logaritmo discreto $e = \log_{b(x)} \bmod n$ è definito solo se b è un generatore del gruppo $\mathbb{Z}_n$. Trovare i generatori di un gruppo, quindi i valori per cui il logaritmo discreto è definito, è a sua volta un problema difficile.

Il numero di radici primitive di $\mathbb{Z}_n^*$ è dato dal **toziente di Eulero**, che equivale al numero di numeri che sono coprimi con n:

$$\phi(n) = |\{i = 1, ..., n \mid \gcd(i, n) = 1\}|$$

3. 7. 1. 1. Algoritmo baby-steps giant-steps

Si tratta di un algoritmo per il calcolo del logaritmo discreto. Ha un costo esponenziale dell'ordine di $O\left(\sqrt{2^n}\right)$, ma il modo migliore per valutarne il costo è il prodotto spazio $\times$ tempo impiegato. L'algoritmo infatti permette di stabilire quale di queste 2 componenti privilegiare.

L'algoritmo parte dalla base g del logaritmo, il suo argomento x e dal modulo p, dopodiché calcola 2 successioni di numeri indipendenti:

- una di queste successioni avrà incrementi piccoli tra un numero e l'altro (baby)
- l'altra li avrà molto più grandi (giant)

L'algoritmo termina quando la successione giant genera un numero già incontrato nella successione baby.

L'algoritmo è il seguente:

1. si scelgono due interi r, s tali che $rs \geq p$;
2. si calcolano le due successioni:
   - baby steps: $g^0, g^1, g^2, ..., g^{r-1}$
   - giant steps: $x, xg^{-r}, xg^{-2r}, ..., xg^{-(s-1)r}$
3. se per un qualche valore i, j risulta che $g^i = xg^{-jr}$, ovvero che $g^{i+jr} = x$, allora significa che $i + jr = \log_{g(x)}$

La correttezza dell'algoritmo è dovuta al fatto che ogni elemento $t \in \mathbb{Z}_p^*$ può essere espresso come $t = j + ir$ per $i = 0, 1, ..., s - 1$ e $j = 0, 1, ..., r - 1$.

L'implementazione dell'algoritmo consiste nel memorizzare in una lookup table i valori della successione baby steps, per poi controllare ad ogni passo se l'elemento corrente della successione giant step è presente nella tabella.

Supponendo che il tempo per la ricerca nella lookup table sia $O(1)$, il tempo d'esecuzione dell'algoritmo è $O(r + s)$, con un tempo minimo di $O(\sqrt{p})$ dato che $rs \geq p$.

Dato che la lookup table memorizza elementi della successione baby, il consumo di spazio è $O(r)$. Non ci sono valori particolari di r, s da cui partire, dunque, in base a come si scelgono, è possibile decidere quanto spazio usare per l'algoritmo: più spazio si usa e più velocemente si troverà la collisione.

4. Diffie-Hellman key-exchange protocol

Il protocollo di Diffie-Hellman permette a due parti di ottenere un **segreto condiviso**, cioè un elemento del gruppo $\mathbb{Z}_p^*$.

1. Alice e Bob si mettono d'accordo su un numero primo p e su una **radice primitiva** g di $\mathbb{Z}_p^*$;
2. Alice sceglie un numero $a \in \mathbb{Z}_p^*$, calcola $x_a = g^a \bmod p$ ed invia x_a a Bob;
3. Bob sceglie un numero $b \in \mathbb{Z}_p^*$, calcola $x_b = g^b \bmod p$ ed invia x_b ad Alice;
4. Alice e Bob, sfruttando il valore ricevuto dalla controparte, calcolano il valore $g^{ab} = (x_a)^b = (x_b)^a$;
5. il valore g^{ab} è il segreto condiviso

4. 1. Efficienza

Il calcolo di g^a, g^b, g^{ab} è molto efficiente, perché l'algoritmo per l'esponenziale modulare ha un costo **logaritmico** nella dimensione dell'esponente.

Non esistono invece algoritmi efficienti per trovare un generatore g di un gruppo $\mathbb{Z}_p^*$. Nella pratica però questo problema viene aggirato:
1. si genera un **safe prime** p;
2. si controlla se un certo **valore fisso** (es. 2, 3 o 5, come fa OpenSSL) è un generatore di $\mathbb{Z}_p$

4. 2. Sicurezza

Per poter risalire al segreto condiviso g^{ab}, Eve deve riuscire a calcolare $a = \log_{g(g^a)} \bmod p$ e $b = \log_{g(g^b)} \bmod p$, ovvero deve calcolare due **logaritmi discreti**.

Computational Diffie-Hellman assumption: Se g, a, b sono scelti a caso in $\mathbb{Z}_p^*$, allora il calcolo di g^{ab} conoscendo soltanto g^a e g^b è computazionalmente intrattabile.

L'assuzione di Diffie-Hellman non è ancora stata dimostrata matematicamente, ma la si ritiene vera.

4. 3. Nota sul segreto condiviso g^{ab}

Il valore $g^{ab} \bmod p$ non può essere utilizzato **direttamente** come chiave per un algoritmo di cifratura simmetrica, perché i suoi bit non rispetano le proprietà di **equiprobabilità** ed **indipendenza** richiesti per una chiave simmetrica.

$g^{ab} \bmod p$ infatti non è una sequenza di bit casuali, ma è un valore di $\mathbb{Z}_p^*$, di conseguenza non tutti i bit sono equiprobabili.

1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
pr(b=1)	0.2	0.4	0.5	0.5

Figura 5: Valori binari dei numeri nell'insieme $\mathbb{Z}_{11}^*$.

Nella pratica, al valore $g^{ab} \bmod p$ viene applicata una **funzione hash crittografica** il cui valore è utilizzato come chiave.

4. 4. Problema dell'autenticazione

Nella sua versione originale, il protocollo DH **non prevede autenticazione**. È quindi vulnerabile ad attacchi di tipo MITM.

Per mitigare questo rischio, Alice e Bob **firmano** i valori x_a, x_b quando li inviano all'altra parte.

4. 5. Chiavi DH effimere

Grazie alla cifratura asimmetrica, piuttosto che iniziare uno scambio di chiavi con il protocollo Diffie-Hellman sarebbe in teoria possibile cifrare una chiave simmetrica direttamente utilizzando la propria coppia di chiavi asimmetriche (come veniva fatto nelle versioni più vecchie del protocollo TLS).

Questo approccio però non garantisce **forward secrecy**: se un attaccante, che si è salvato precedentemente tutti i messaggi cifrati, riesce in un qualche modo ad ottenere la chiave privata di una delle due parti, questi può mettere in chiaro **tutti** i messaggi che le due parti si sono scambiate fino a quel momento.

Per mitigare questo rischio si utilizza il protocollo Diffie-Hellman con delle **chiavi effimere**: dopo che sono stati utilizzati una volta, i valori a e b vengono buttati via. In questo modo l'attaccante se anche riuscisse a compromettere le chiavi RSA di una delle due parti, non riuscirebbe comunque a mettere in chiaro i messaggi.

5. Casualità e algoritmi probabilistici

Le sequenze di bit **casuali** rilevanti in ambito crittografico devono avere due proprietà:
1. **ogni** bit deve avere probabilità pari ad $\frac{1}{2}$ di essere 0 o 1;
2. i bit devono essere **indipendenti** tra di loro

L'unico modo per generare sequenze di bit **realmente** casuali è tramite l'uso di **generatori hardware** che sfruttano fenomeni fisici stocastici.

Questi generatori però sono molto costosi e poco flessibili, perché è difficile generare in breve tempo sequenze di bit di **lunghezza arbitraria**.

Per questi motivi sono stati sviluppati diversi **algoritmi software** per generare sequenze di bit **pseudo**-casuali. Tutti questi algoritmi prevedono in input un parametro, detto **seed**, che è la sorgente di casualità. I CSPRNG sono algoritmi **deterministici**: eseguendo l'algoritmo con lo stesso seed si ottiene lo stesso output.

Esempi di sorgenti di casualità sono la velocità di battitura sulla tastiera oppure i movimenti del mouse.

I vari sistemi operativi raccolgono le varie sequenze di casualità all'interno di un **entropy pool**. Quando un'applicazione ha bisogno di bit casuali, il kernel li preleva da questo file.

5. 1. Algoritmi probabilistici

Un algoritmo si dice **probabilistico** se al suo interno utilizza una **sorgente di casualità** che fornisce sequenze di bit indipendenti e con probabilità uniforme.

Per valutare la complessità di questi algoritmi si usa il modello **bit cost**: il costo di un'operazione logico/aritmetica dipende dal **numero di bit** dei suoi operandi.

Nella valutazione di un algoritmo probabilistico bisogna tenere in considerazione anche la **quantità di bit casuali richiesti**, dato che non sono una risorsa infinita.

5. 1. 1. Algoritmi di tipo Monte Carlo

Sono algoritmi di tipo **decisionale** che vengono chiamati anche **probability-bounded one-sided error**:
- probability bounded perché la probabilità che la risposta sia sbagliata è > 0, ma è **limitata** e **non dipende dall'input**;
- one-sided error perché l'algoritmo può sbagliare **solo su una** delle due possibili risposte

5. 1. 1. 1. Esempio di algoritmo NON è probability-bounded

```
IsPrime(n):
if n mod 2 = 0:
    return False
choose an odd p ∈ [2, n/2] randomly
if n mod p = 0:
    return False
return True
```

Figura 6: Algoritmo probabilistico per determinare se n è primo

input	output
• se n è primo, l'algoritmo restituisce sempre la risposta corretta; • se invece n è composto, la risposta dell'algoritmo potrebbe essere sbagliata	• se la risposta è `False`, allora è sempre quella corretta; • se invece la risposta è `True`, potrebbe essere sbagliata perché potrebbe essere dovuta alla sfortuna nella scelta di p

Tabella 3: Dimostrazione del fatto che l'algoritmo è one-sided error, sia dal lato degli input che dal lato degli output

L'algoritmo però non è probability bounded.

Per dimostrarlo, si consideri che per ogni divisore x di n anche $\frac{n}{x}$ è un divisore di n. Uno fra questi due divisori è $< \sqrt{n}$, dunque n può avere al massimo $2\sqrt{n}$ divisori. Scegliendo un p dispari nell'intervallo $\left[2, \frac{n}{2}\right]$, la probabilità che p sia un divisore di n è data da:

$$\frac{2\sqrt{n}}{\frac{n}{4}} = \frac{8}{\sqrt{n}} \underset{n\to\infty}{\longrightarrow} 0$$

Dunque la probabilità che la risposta sia giusta **diminuisce sempre di più** al crescere di n.

5. 1. 1. 2. Importanza del limite sull'errore probabilistico

Se la probabilità che l'algoritmo dia la risposta sbagliata è **limitata superiormente**, **non dipende dall'input** e le run dell'algoritmo sono **indipendenti** tra di loro, allora è possibile aumentare arbitrariamente la probabilità che la risposta sia corretta semplicemente eseguendo lo stesso algoritmo con lo stesso input per più volte.

Esempio: se la probabilità di risposta giusta di un algoritmo è dell'1%, la probabilità che dopo k run con lo stesso input la risposta continui ad essere sbagliata è 0.99^k. Se l'algoritmo viene eseguito circa 70 volte, la probabilità che la risposta sia ancora sbagliata è "solo" del 50%:

$$0.99^k \leq \frac{1}{2} \Longrightarrow k \leq \frac{\ln\left(\frac{1}{2}\right)}{\ln(0.99)} \approx 69$$

5. 1. 2. Algoritmi di tipo Las Vegas

Gli algoritmi di tipo Las Vegas restituiscono **sempre la risposta corretta**, ma ad essere probabilistico è il **tempo** con cui la trovano. Un algoritmo decisionale di tipo Las Vegas risponde quasi sempre "non so", ma quando restituisce una delle due possibili risposte allora è sempre quella giusta.

5. 1. 2. 1. Esempio: algoritmo per aggiustare l'uniformità di una sequenza casuale

Si supponga di avere una sequenza $\{z_i\}_{i\geq 1}$ di bit casuali indipendenti ma non uniformi:

$$\mathbb{P}[z_i = 0] = p \quad \mathbb{P}[z_i = 1] = 1 - p \quad 0 < p < 1$$

```
FixUniformity(z, n):
for i = 1 to n, i = i + 2:
    if z_i ≠ z_{i+1}:
        return z_i
```

Figura 7: Algoritmo di tipo Las Vegas per aggiustare l'uniformità dei bit di z

La probabilità che l'algoritmo restituisca 0 è pari a $\mathbb{P}[z_i = 0] \cdot \mathbb{P}[z_{i+1} = 1] = p(1-p)$, che equivale alla probabilità che l'algoritmo restituisca 1. Dunque quando l'algoritmo restituisce una risposta, questa è sempre corretta, perché entrambe le risposte sono equiprobabili.

Invece la probabilità che l'algoritmo non restituisca niente è pari alla probabilità che $z_i = z_{i+1}$:

$$\mathbb{P}[z_i = 0] \cdot \mathbb{P}[z_{i+1} = 0] + \mathbb{P}[z_i = 1] \cdot \mathbb{P}[z_{i+1} = 1] = 2p(p-1) + 1$$

dunque se $s = 2p(p-1) + 1$, la probabilità che il risultato sia restituito al k-esimo tentativo è data da $s^{k-1}(1-s)$, che coincide con la **serie geometrica**. Conoscendo p, il numero di volte in cui dev'essere eseguito l'algoritmo è:

$$\sum_{k=1}^{\infty} ks^{k-1}(1-s) = \frac{1}{1-s} = -\frac{1}{2p(p-1)}$$

Questo risultato si può sfruttare anche per fare il ragionamento inverso: qual è la probabilità che $2k$ bit siano sufficienti per ottenere il risultato?

$$\sum_{i=1}^{k} s^{i-1}(1-s) = 1 - s^k$$

5. 2. Numeri primi e test di primalità

Prime number theorem: se $\pi(n)$ è il numero di numeri primi $\leq n$, allora:

$$\pi(n) \sim \frac{n}{\ln(n)}$$

Questo implica che nell'intervallo $[2, n]$ ci sono circa $\frac{1}{\ln(n)}$ numeri primi, ovvero i numeri primi non sono poi tanto rari.

Il prime number theorem ci dice che per trovare un numero primo scegliendo a caso nell'intervallo $[2, n]$ occorrono circa $n \ln(2) \approx 0.69 \cdot n$ tentativi.

5. 2. 1. Test di primalità basato sul piccolo teorema di Fermat

Una prima idea per un test di primalità potrebbe essere quella di sfruttare il **piccolo teorema di Fermat**:

Piccolo teorema di Fermat: se p è primo, allora vale la seguente relazione:

$$x^{p-1} \bmod p = 1$$

per ogni x tale che $\gcd(x, p) = 1$ (cioè per ogni $0 < x < p$, dato che p è primo)

Questo teorema però non vale al contrario: se $x^{n-1} \bmod n = 1$, non è detto che n sia primo. Un numero n composto per cui risulta che $x^{n-1} \bmod n = 1$ è detto **pseudo-primo base n**. L'esistenza di questa categoria di numeri impedisce di utilizzare l'FLT come criterio di primalità.

Tuttavia si può dimostrare che gli pseudo-primi base 2, ad esempio, sono **estremamente rari**. Se quindi n è scelto a caso e si pone $x = 2$, è molto improbabile che $2^{n-1} \bmod n = 1$.

```
FERMAT-PRIMALITY-TEST(n):
return (2^(n-1) mod n) == 1
```

Figura 8: Algoritmo per stabilire se n è primo basato sul piccolo teorema di Fermat

Questo è un algoritmo one-sided error:
- se n è primo, il risultato è sempre True;
- se invece n è composto, il risultato potrebbe essere o True o False

La probabilità d'errore dell'algoritmo si può ulteriormente abbassare provando **più di una base**. L'idea è che se n è pseudo-primo base 2, è improbabile che lo sia anche base 3 e così via.

```
IMPROVED-FERMAT-PRIMALITY-TEST(n):
pick a randomly in {3, ..., n − 1}
if gcd(a, n) > 1:
    return False
return (2^(n−1) mod n) == 1 and (a^(n−1) mod n) == 1
```

Figura 9: Versione migliorata del test di primalità basato sul piccolo teorema di Fermat

In generale, questa versione dell'algoritmo è più efficace perché:
1. aggiunge un test sul GCD, che con probabilità > 0 potrebbe individuare un divisore di n;
2. esegue il test basato sull'FLT con 2 basi anziché con una sola

Esistono però dei numeri composti, detti **numeri di Charmichael**, per cui per **qualunque** a tale che $\gcd(a, n) = 1$ risulta che $a^{n-1} \bmod n = 1$.

Sebbene i numeri di Charmichael siano estremamente rari (ancora più degli pseudo-primi base 2), questo algoritmo non è comunque considerabile un algoritmo Monte Carlo perché non usa nessuna **sorgente di casualità**: se la risposta dell'algoritmo è True è inutile rieseguirlo più volte per vedere se la risposta cambia, perché non lo farà mai.

5. 2. 2. Test di primalità di Solovay-Strassen

Residuo quadratico: un elemento $a \in \mathbb{Z}_n$ è detto **residuo quadratico** modulo n se esiste un $x \in \mathbb{Z}_p$ tale che $a = x^2 \bmod n$.

L'insieme dei residui quadratici forma un **sottogruppo** di $\mathbb{Z}_n^*$ il cui ordine, se n è primo, è pari alla metà dell'ordine di $\mathbb{Z}_n^*$. Il sottogruppo dei residuo quadratici è formato da tutte le **potenze pari** di un generatore g.

Simbolo di Legendre: se p è un numero primo ed $a \in \mathbb{Z}_p$, il **simbolo di Legendre** è una funzione definita come:

$$\left(\frac{a}{p}\right) = \begin{cases} 0 \text{ se} \gcd(a, p) > 1 \\ 1 \text{ se } a \text{ è un residuo qudratico} \bmod p \\ -1 \text{ se } a \text{ non è un residuo quadratico} \bmod p \end{cases}$$

Il simbolo di Legendre può essere calcolato con il **criterio di Eulero**:

$$\left(\frac{a}{p}\right) = a^{\frac{p-1}{2}} \bmod p$$

Dunque per capire se a è un residuo quadratico modulo p è sufficiente calcolare un'esponenziale modulare.

Il simbolo di Legendre consente di verificare agevolmente se 2 o 5 sono dei residui quadratici:
- 2 è un residuo quadratico modulo p se e solo se $p \equiv 1, 7 (\bmod 8)$, perché vale la seguente proprietà:

$$\left(\frac{2}{p}\right) = (-1)^{\frac{p^2-1}{8}}$$

- 5 è un residuo quadratico modulo p se e solo se $p \equiv 1, 4 (\bmod 5)$

Queste due proprietà vengono utilizzate da OpenSSL durante la fase di generazione dei parametri per Diffie-Hellman.

Simbolo di Jacobi: sia n un numero composto e sia $p_1 \cdot p_2 \cdot ... \cdot p_k$ la sua **scomposizione in fattori primi**. Per ogni intero a il simbolo di Jacobi è definit come:

$$\left(\frac{a}{n}\right) = \prod_{i=1}^{k} \left(\frac{a}{p_i}\right)$$

Il simbolo di Jacobi è una generalizzazione del simbolo di Legendre che vale anche per numeri composti. A differenza del simbolo di Legendre, se $\left(\frac{a}{n}\right) = 1$ non si può dire che a è un residuo quadratico modulo n, perché potrebbe capitare una cosa di questo tipo:

$$\left(\frac{2}{15}\right) = \left(\frac{2}{3}\right)\left(\frac{2}{5}\right) = (-1)^2 = 1$$

Se invece $\left(\frac{a}{n}\right) = -1$ allora si può dire che a **non** è un residuo quadratico modulo n.

Se invece a è un residuo quadratico modulo n, allora:
- se $\gcd(a, n) > 1$ può succedere che $\left(\frac{a}{n}\right) = 0$
- altrimenti $\left(\frac{a}{n}\right) = 1$

Il simbolo di Jacobi gode di diverse proprietà che permettono di calcolarlo **senza passare per la fattorizzazione di n**.

```
IsPrime(n):
pick a ∈ {2, ..., n − 1} randomly
if gcd(a, n) > 1:
    return False
J = (a/n)                          // Jacobi's symbol
P = a^((p−1)/2) mod n              // Euler's criterion
return J == P
```

Figura 10: Test di primalità di Solovay-Strassen

L'idea di base del test di primalità di Solovay-Strassen è che:
- se $J \neq P$, allora sicuramente n è composto, perché significa che il simbolo di Jacobi è diverso da quello di Legendre (calcolato con il criterio di Eulero), mentre sarebbero uguali se n fosse primo;
- se $J = P$ allora n **potrebbe** essere primo

Un numero composto n che soddisfa l'uguaglianza $J = P$ per un qualche valore $a \in \{2, ..., n-1\}$ viene detto **pseudo-primo di Eulero** rispetto alla base a. Un numero non può essere pseudo-primo di Eulero rispetto a **tutte** le possibili basi a, dunque ripetendo più volte l'algoritmo è possibile abbassare arbitrariamente la probabilità che la risposta sia sbagliata.

Il test di Solovay-Strassen è un algoritmo di tipo Monte Carlo, perché:
- è **one-sided error**: l'algoritmo può restituire la risposta sbagliata solo se n è composto;

- è **probability-bounded**: Solovay e Strassen hanno dimostrato che per ogni $a \in \{2, ..., n-1\}$ esistono $\frac{n}{2}$ valori di a che sono testimoni della non primalità di n. Su input n composto, quindi, l'algoritmo sbaglia con probabilità $\leq \frac{1}{2}$.

5. 3. Fattorizzazione di interi

La fattorizzazione di numeri interi è un altro **problema difficile** che sta alla base di alcuni cifrari asimmetrici (ad esempio RSA e Rabin).

Un algoritmo di tipo brute-force che prova tutti i numeri nell'intervallo $[2, \sqrt{n}]$ per cercare un fattore di n ha costo **esponenziale** rispetto alla dimensione di n.

Il miglior algoritmo noto per la fattorizzazione è il **General Number Field Sieve**, che ha un costo **sub-esponenziale** (più di polinomiale ma meno di esponenziale).

5. 3. 1. Algoritmo di fattorizzazione ρ di Pollard

Si tratta di un **algoritmo iterativo**, di **costo esponenziale**, che permette di trovare un fattore di n.

L'idea dell'algoritmo è di costruire una sequenza di valori x_j che sembra apparentemente casuale, ma che ad un certo punto **inizia a ripetersi**.

Ogni 2^t iterazioni, con $t = 0, 1, ...$ che cresce di continuo, il valore dell'iterata corrente x_j viene memorizzato dall'algoritmo. Se per una qualche iterazione il valore dell'iterata corrente x_j è pari a questo valore salvato y, allora è stato trovato un fattore.

```
POLLARD-RHO(n, f):
i = 2
j = 1
pick x ∈ {0, ..., n − 1} randomly
y = x                                   // y is the stored value
while True:
    x = f(x)
    m = gcd(y − x, n)
    if m > 1 and m ≠ n:
        return m
    j = j + 1
    if j == i:
        y = x
        i = 2i
```

Figura 11: Algoritmo ρ di Pollard per la fattorizzazione

f è una funzione che ha lo scopo di generare valori (apparentemente) casuali all'interno di $\mathbb{Z}_p$. Le due funzioni che si utilizzano di più nella pratica sono $f(x) = (x^2 + 1) \bmod n$ ed $f(x) = (x^2 - 1) \bmod n$.

Di solito si preferisce scegliere $x = 2$ come valore iniziale, anziché scegliere un valore a caso in $\mathbb{Z}_n$. Nella pratica però questo algoritmo è utilizzato in **contesti multi-thread**: thread diversi eseguono l'algoritmo partendo da valori iniziali diversi con la speranza che un qualche thread riesca a trovare un fattore di n.

L'algoritmo funziona perché per ogni termine della successione $x_i = f(x) \bmod n$, esiste un termine di un'altra successione "ombra" $x'_i = x_i \bmod p$, dove p è un fattore non banale di n.

La successione "ombra" ha le stesse proprietà della successione degli x_i:

$$\begin{aligned} x'_{i+1} &= x_{i+1} \bmod p \\ &= (x_i^2 - 1 \bmod n) \bmod p \\ &= (x_i^2 - 1) \bmod p \\ &= \left((x_i \bmod p)^2 - 1\right) \bmod p \\ &= \left((x'_i)^2 - 1\right) \bmod p \end{aligned}$$

Supponendo che la funzione f sia una funzione casuale (cosa che in realtà non è vera, ma è coerente con il comportamento dell'algoritmo), si può dire che ogni valore della successione x_i è uniformemente distribuito nell'insieme $\mathbb{Z}_n$. Essendo questo un insieme finito, per il **birthday paradox** si può dire che servono $\Theta(\sqrt{n})$ iterazioni dell'algoritmo per ri-ottenere uno stesso valore di x_i.

Seguendo lo stesso ragionamento per la successione x'_i, che è una successione di valori in $\mathbb{Z}_p$, servono $\Theta(\sqrt{p})$ iterazioni per trovare una collisione tra due valori della successione. Poiché l'algoritmo termina quando i due valori x ed y sono congruenti modulo p, il costo computazionale dell'algoritmo è proprio $O(\sqrt{p}) = O\left(\sqrt{\sqrt{n}}\right) = O(\sqrt[4]{n})$, dato che $p < \sqrt{n}$.

5. 3. 1. 1. Casi sfavorevoli

Ci sono alcuni casi sfavorevoli in cui l'algoritmo non si accorge di avere un fattore di n già "in mano", per cui continua ad eseguire delle altre iterazioni.

Un caso lo si ha quando $n = p \cdot q$ con p e q primi. L'algoritmo arriverà ad un punto in cui x ed y sono **entrambi** congruenti sia modulo p che modulo q e dunque lo sono anche modulo n per il CRT, perciò risulterà che $\gcd(y - x, n) = n$.

Un altro caso simile lo si ha quando $n = p^k$, dove risulterà che $y = x$ e dunque $\gcd(y - x, n) = 0$.

Nella pratica questi casi sfavorevoli sono trascurabili. L'algoritmo è comunque in grado di trovare un fattore in questi casi, semplicemente ci vorrà più tempo a causa di queste iterazioni "sprecate".

6. RSA

Storicamente, RSA è stato il primo cifrario asimmetrico realizzato.

Generazione della chiave:
1. Alice sceglie la **dimensione in bit** della chiave N;
2. Alice sceglie a caso due **numeri primi** p, q, entrambi di dimensione $\frac{N}{2}$;
3. Alice calcola $n = pq$ e $\phi(n) = (p-1)(q-1)$;
4. Alice sceglie un numero intero e tale che $\gcd(e, \phi(n)) = 1$;
5. Alice calcola $d = e^{-1} \bmod \phi(n)$;

La coppia (n, e) è la chiave pubblica di Alice, mentre d è la chiave privata.

Per cifrare un messaggio M, con $M \in [0, n-1]$, Alice calcola $C = M^e \bmod n$, utilizzando l'esponente e di Bob. Per decifrare il messaggio, Bob calcola $M = C^d \bmod n$.

6. 1. Dimostrazione della correttezza

Dimostrare la correttezza di RSA significa dimostrare che $(M^e)^d \bmod n = M$.

Innanzi tutto si sfrutta il fatto che $(ed) \bmod \phi(n) = 1$, ovvero $ed = k \cdot \phi(n) + 1$ per un qualche intero k. Dopodiché si riscrive l'espressione in questo modo:

$$\begin{aligned}(M^e)^d \bmod n &= \left[M^{k \cdot \phi(n)+1}\right] \bmod n \\ &= \left[M \cdot M^{k(p-1)(q-1)}\right] \bmod n\end{aligned}$$

Il problema poi diventa dimostare queste uguaglianze:

$$\left(M \cdot M^{k(p-1)(q-1)}\right) \bmod p = M \bmod p \qquad \left(M \cdot M^{k(p-1)(q-1)}\right) \bmod q = M \bmod q$$

Se queste uguaglianze sono vere, allora per il **teorema cinese dei resti** sarà vero anche che $\left(M \cdot M^{k(p-1)(q-1)}\right) \bmod n = M \bmod n$.

Per dimostrare l'uguaglianza con p si distinguono 2 casi:
1. $M \bmod p = 0$
2. $M \bmod p \neq 0$

Nel 1° caso p è un divisore di M, dunque l'uguaglianza è sicuramente dimostrata perché entrambi i membri valgono 0:

$$M \bmod p = 0 \iff \left(M \cdot M^{k(p-1)(q-1)}\right) \bmod p = 0$$

Anche il 2° caso è immediato grazie al piccolo teorema di Fermat:

$$\begin{aligned}\left(M \cdot M^{k(p-1)(q-1)}\right) \bmod p &= \left[M \cdot \left(M^{(p-1)^{k(q-1)}}\right)\right] \bmod p \\ &= \left[M \cdot \left(M^{p-1} \bmod p\right)^{k(q-1)}\right] \bmod p \\ &= M \bmod p\end{aligned}$$

Dunque anche in questo caso l'uguaglianza è verificata.

Per dimostrare l'uguaglianza con q è sufficiente ripetere gli stessi passaggi.

6. 2. Efficienza di RSA

Considerato che:

- non sono richiesti requisiti particolari per i primi p e q (in particolare non è richiesto che siano dei **safe prime**);
- il calcolo dell'esponenziale modulare è molto efficiente

si può dire che RSA sia piuttosto efficiente.

Il problema di trovare e tale che $\gcd(e, \phi(n)) = 1$ viene girato al contrario: si tiene un valore fisso di e e si generano due numeri primi p e q finché $\gcd(e, (p-1)(q-1))$ non risulta uguale ad 1.

6. 3. Sicurezza di RSA

La sicurezza di RSA sta nella **difficoltà di fattorizzare numeri molto grandi**. Se questo problema fosse facile, allora lo sarebbe anche il violare RSA.

Non è (ancora) stato dimostrato che invertire la funzione di cifratura di RSA sia equivalente alla fattorizzazione, ma lo si ritiene vero.

6. 3. 1. Malleabilità

La versione textbook di RSA è **malleabile**.

Sia $C_1 = (M_1)^e$ il messaggio da decifrare.

L'attacco che si può fare è il seguente:

1. si costruire un **ciphertext intermedio** $C_I = 2^e \bmod n$;
2. si calcola $C_2 = C_1 \cdot C_I$;
3. si richiede una decifratura di C_2, ottenendo M_2;

Per risalire ad M_1 è sufficiente calcolare $\frac{M_2}{2}$, perché:

$$\begin{aligned} M_2 = (C_2)^d \bmod n &= (C_1 \cdot C_I)^d \bmod n \\ &= [(M_1^e \bmod n) \cdot (2^e \bmod n)]^d \bmod n \\ &= (M_1 \cdot 2)^{ed} \bmod n \\ &= 2M_1 \bmod n \end{aligned}$$

Per mitigare questa vulnerabilità si utilizzano delle tecniche di **hashing**.

6. 3. 2. e troppo piccolo

Se e è troppo piccolo, c'è il rischio che $M^e < n$ e che quindi $C = M^e \bmod n = M^e$. In questo caso per ottenere M basterebbe calcolare la **radice e-esima di C**, senza doversi impegnare nella fattorizzazione di n.

Per questa ragione l'attuale standard implementativo prevede di scegliere sempre $e = 2^{16} + 1 = 65.537$, in quanto è sufficientemente grande da far lavorare sempre il modulo.

6. 3. 3. Algoritmo di fattorizzazione di Fermat

Questo algoritmo permette di semplificare la fattorizzazione di n quando p e q sono **sufficientemente vicini** tra loro. In binario, il "sufficientemente vicini" significa che le **metà più significative** di p e q sono uguali.

L'algoritmo parte considerando il fatto che il prodotto di due numeri interi dispari può essere espresso come **differenza di quadrati**:

$$n = a^2 - b^2 = \left(\underbrace{\frac{p+q}{2}}_{a}\right)^2 - \left(\underbrace{\frac{p-q}{2}}_{b}\right)^2$$

a^2 e b^2 sono dei **quadrati perfetti**, infatti:

$$n = a^2 - b^2 \iff a = \sqrt{b^2 + n} \iff b = \sqrt{n - a^2}$$

Dunque si può tentare un approccio **bruteforce** per trovare un quadrato perfetto:

<u>FERMAT-FACTOR</u>$(n,\ x_0)$:
$z_i = x_0^2 - n$
if z_i is a perfect square**:**
    return $(p = x_0 + \sqrt{z}, q = x_0 - \sqrt{z})$
return Fermat-Factor$(n, x_0 + 1)$

Figura 12: Algoritmo di fattorizzazione di Fermat

6. 3. 3. 1. Scelta del punto di partenza

Da quale valore x_0 partire nell'algoritmo?

Partendo dalla seguente uguaglianza:

$$\left(\frac{p+q}{2}\right)^2 - n = \left(\frac{p-q}{2}\right)^2$$

dato che l'algoritmo incrementa sempre il valore di x_0, sicuramente per partire bisogna scegliere un valore $\leq \frac{p+q}{2}$. Non conoscendo p e q però in realtà quest'informazione è poco utile all'algoritmo (non saprebbe comunque quando fermarsi).

Tuttavia si può dimostrare che se $n = p \cdot q$ con $p \neq q$, allora $\frac{p+q}{2} > \sqrt{n}$. Per farlo è sufficiente una **dimostrazione per assurdo**:

$$\frac{p+q}{2} \leq \sqrt{n} \longrightarrow \frac{1}{4}(p^2 + q^2) + pq \leq n$$

il che è falso, dunque $\frac{p+q}{2} > \sqrt{n}$.

Un buon punto di partenza quindi è $x_0 = \lceil \sqrt{n} \rceil$.

6. 3. 3. 2. Efficienza dell'algoritmo

Sia $p > q$ e dunque $q < \sqrt{n} < p$ (il ragionamento è analogo nel caso in cui $q > p$).

Si supponga che la **metà più significativa** dei bit di p coincida con quella di q, ovvero vale la seguente relazione:

$$\frac{p-q}{2} < c\sqrt{2q}$$

per un qualche valore di c.

L'uguaglianza da cui parte l'algoritmo di Fermat può essere riscritta in questo modo:

$$\left(\frac{p+q}{2}\right)^2 - n = \left(\frac{p-q}{2}\right)^2$$
$$\Downarrow$$
$$\left(\frac{p+q}{2} + \sqrt{n}\right)\left(\frac{p+q}{2} - \sqrt{n}\right) = \left(\frac{p-q}{2}\right)^2$$

Dato che $q < \sqrt{n} < p$ per ipotesi, vale che $2q < \frac{p+q}{2} + \sqrt{n}$ e dunque si può passare a questa disuguaglianza:

$$2q\left(\frac{p+q}{2} - \sqrt{n}\right) < 2qc^2 \longrightarrow \frac{p+q}{2} - \sqrt{n} < c^2$$

Questo dimostra che il **numero di passi** che l'algoritmo deve eseguire, che è proprio $\frac{p+q}{2} - \sqrt{n}$, è **limitato superiormente** da c^2.

Dato che $q = O(\sqrt{n})$ e $q < \sqrt{n} < p$ per ipotesi, l'assunzione iniziale $\frac{p-q}{2} < c\sqrt{2q}$ implica che $p - q = O(\sqrt[4]{n})$, ovvero che p e q coincidono nella loro **metà più significativa**.

6. 3. 4. Riutilizzo di p e q

Riutilizzare lo stesso valore di p (o di q) per due moduli distinti n_1, n_2 rende il processo di cifratura totalmente inutile, perché l'attaccante riesce a recuperare in un colpo solo **entrambe** le chiavi private calcolando il GCD delle due chiavi pubbliche:

<u>RSA-CommonFactor</u>$((e_1, n_1), (e_2, n_2))$:
$p = \gcd(n_1, n_2)$
$q_1 = \frac{n_1}{p}$
$q_2 = \frac{n_2}{p}$
$d_1 = e_1^{-1} \bmod (p-1)(q_1 - 1)$
$d_2 = e_2^{-1} \bmod (p-1)(q_2 - 1)$
return (d_1, d_2)

Figura 13: Algoritmo per rompere RSA in caso di riuso di p o q

6. 4. Aspetti implementativi di RSA

6. 4. 1. Velocizzare la decifratura

Sfruttando il CRT si può rendere più efficiente il processo di cifratura arrivando a fare a meno della chiave privata d.

Si considerino le seguenti quantità:

$$M_p = C^d \bmod p \quad M_q = C^d \bmod q$$

Dato che $d = e^{-1} \bmod (p-1)(q-1)$, si può "scorporare" in due valori s, t tali che:

$$s = d \bmod (p-1) \qquad t = d \bmod (q-1)$$

Dalla **definizione di resto** segue che:

$$d = s + a(p-1) \qquad d = t + b(q-1)$$

per una qualche coppia di interi a e b.

Le due quantità $C^d \bmod p$ e $C^d \bmod q$ possono quindi essere riscritte in questo modo:

$$\begin{aligned} C^d \bmod p &= C^{s+a(p-1)} \bmod p \\ &= \left[C^s \cdot \left(C^{p-1}\right)^a\right] \bmod p \\ &= C^s \bmod p \end{aligned} \qquad \begin{aligned} C^d \bmod q &= C^{t+b(q-1)} \bmod q \\ &= \left[C^t \cdot \left(C^{q-1}\right)^b\right] \bmod q \\ &= C^t \bmod q \end{aligned}$$

da cui segue che

$$M_p = C^s \bmod p \qquad M_q = C^t \bmod q$$

Per ottenere M è sufficiente rimettere insieme i pezzi con il CRT:

$$M = \left[q(q^{-1} \bmod p)M_p + p(p^{-1} \bmod q)M_q\right] \bmod n$$

Questo metodo richiede il calcolo di 4 valori aggiuntivi rispetto alla versione "classica":
- $C^s \bmod p$ e $C^t \bmod q$, da calcolare per ogni messaggio;
- $q(q^{-1} \bmod p)$ e $p(p^{-1} \bmod q)$, da calcolare una sola volta

ma nonostante questi conti aggiuntivi, questo metodo è **fino a quattro volte più efficiente** rispetto alla decifratura classica perché permette di lavorare con esponenti s e t di dimensione **dimezzata** rispetto a quella di d.

6. 5. Optimal Asymmetric Encryption Padding

La **malleabilità** in RSA è dovuta al fatto che il processo di cifratura è **deterministico**. Lo schema di padding OAEP ha lo scopo di inserire degli **elementi casuali** nel processo al fine di irrobustirlo.

Parametri dello schema:
- la dimensione in bit del modulo (N);
- una sequenza r di **bit casuali**, di lunghezza k;
- il numero di **bit di padding** h che s'intende utilizzare;
- due **funzioni hash crittografiche** G ed H;
- una sequenza di bit P composta da soli zeri, da appendere in fondo al messaggio, di lunghezza h;

<u>RSA-OAEP-Pad</u>(m, P, r, G, H):
$m_1 = G(r) \oplus (m \parallel P)$
$r_1 = H(m_1) \oplus r_1$
return $x = 00 \parallel m_1 \parallel r_1$

<u>RSA-OAEP-Unpad</u>(x):
split x to get m_1 and r_1
$r = H(m_1) \oplus r_1$
$m = G(r) \oplus m_1$
if last h bits of m are not 0**:**
    throw error
remove last h bits of m
return m

Tabella 4: Padding e unpadding con OAEP

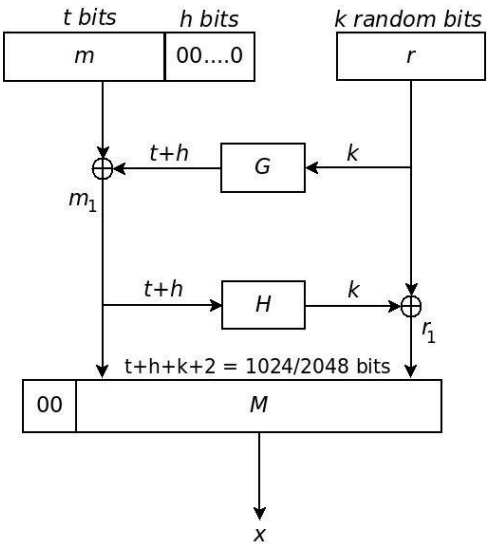

Figura 14: Schema OAEP

7. Protocollo di Rabin

Si tratta di un algoritmo che non è mai stato utilizzato nella pratica, ma che è comunque interessante dal punto di vista teorico perché è dimostrato che invertire la funzione di cifratura è **equivalente** alla fattorizzazione di interi (dimostrazione che invece non esiste per RSA).

Il problema che ha impedito l'uso pratico del protocollo è per 1 ciphertext l'algoritmo di decifratura restituisce **4** possibili plaintext, quindi è necessario aggiungere della complessità all'algoritmo per capire quale delle 4 alternative è quella corretta.

Risalire all'alternativa corretta può essere fatto solo per messaggi di **testo**; non si può usare Rabin per cifrare **sequenze di bit** arbitrari perché poi il destinatario non sarebbe in grado di decifrarle.

7. 1. Calcolo delle radici quadrate modulari

Sia $n = p \cdot q$ (con p e q primi) e sia $y \in \mathbb{Z}_n$ un valore di cui si vuole calcolare la **radice quadrata**.

Il problema può essere approcciato in senso opposto: siano $x \in \mathbb{Z}_n$ ed $y = x^2 \bmod n$. Avendolo definito così, una delle radici di y è nota, ed è esattamente x. Come nel caso reale, anche l'**opposto modulare** di x è una radice di y, dunque si ha:

$$\sqrt{y} = \pm x \bmod n$$

Dato però che $n = p \cdot q$, le radici di y non sono 2, ma 4. $y \bmod p$ ed $y \bmod q$ sono a loro volta dei **residui quadratici** modulo p e modulo q rispettivamente, ed $x \bmod p$ ed $x \bmod q$ sono due delle loro radici:

$$\begin{aligned}
[(x \bmod p) \cdot (x \bmod p)] \bmod p &= (x \cdot x) \bmod p \\
&= [(x \cdot x) \bmod n] \bmod p \\
&= y \bmod p
\end{aligned}$$

Le 4 radici di $y \bmod n$ possono essere calcolate quindi in questo modo:

$$\begin{aligned}
r_1 &= (c_1 z_1 + c_2 w_1) \bmod n \\
r_2 &= (c_1 z_1 - c_2 w_1) \bmod n \\
r_3 &= (-c_1 z_1 + c_2 w_1) \bmod n \\
r_4 &= (-c_1 z_1 - c_2 w_1) \bmod n
\end{aligned}
\qquad
\begin{aligned}
c_1 &= q(q^{-1} \bmod p) \\
c_2 &= p(p^{-1} \bmod q)
\end{aligned}$$

per un qualche valore $z_1 \in \mathbb{Z}_p$ e $w_1 \in \mathbb{Z}_q$. Questi risultati valgono anche se si prendono gli **opposti modulari** di z_1 e w_1.

Due di queste 4 radici sono congrue modulo p, mentre le altre lo sono modulo q.

7. 2. Cifratura e decifratura

Si scelgono due numeri primi p, q tali che $(p, q) \equiv 3 (\bmod 4)$, poi si calcola $n = p \cdot q$.

Dato il messaggio in chiaro M, il messaggio cifrato si calcola come $C = M^2 \bmod n$.

La decifratura consiste nel calcolare le 4 radici quadrate modulo n di C. Uno di questi 4 valori è il messaggio in chiaro.

Per trovare le radici, si parte considerando le due quantità

$$M_p = C^{\frac{p+1}{4}} \qquad M_q = C^{\frac{q+1}{4}}$$

M_p ed M_q sono delle radici quadrate di C modulo p e modulo q rispettivamente, perché:

$$\begin{aligned} M_p^2 \bmod p &= \left(C^{\left(\frac{p+1}{4}\right)^2}\right) \bmod p \\ &= \left(C \cdot C^{\frac{p-1}{2}}\right) \bmod p \\ &= C \bmod p \end{aligned}$$

e analogo per M_q^2.

Siano ora $C_p = q(q^{-1} \bmod p)$ e $C_q = p(p^{-1} \bmod q)$. Vale quanto segue:

$$\left(C_p M_p + C_q M_q\right) \bmod p = M_p \qquad \left(C_p M_p + C_q M_q\right) \bmod q = M_q$$

dunque per il CRT significa che $C_p M_p + C_q M_q$ è una delle 4 radici di $C \bmod n$.

Le altre radici si ottengono combinando i segni:

$$\begin{aligned} \left(C_p M_p + C_q M_q\right) \bmod n \\ \left(C_p M_p - C_q M_q\right) \bmod n \\ \left(-C_p M_p + C_q M_q\right) \bmod n \\ \left(-C_p M_p - C_q M_q\right) \bmod n \end{aligned}$$

7. 3. Dimostrazione dell'equivalenza alla fattorizzazione

```
RABIN-FACTOR(n):
pick a random M such that 0 ≤ M < n
M_i = Rabin-Decrypt(M^2 mod n)
m = gcd(n, M − M_i)
if m > 1 and m ≠ n:
    return m
else:
    pick another random M and try again
```

Figura 15: Algoritmo di tipo Las Vegas per trovare un fattore di n con l'algoritmo di Rabin

Supponendo che il messaggio reale sia $M = \left(C_p M_p + C_q M_q\right) \bmod n$:

- se Rabin restituisce $M_i = \left(C_p M_p + C_q M_q\right) \bmod n$, allora $M - M_i = 0$ e dunque $\gcd(n, M - M_i) = 0$, quindi non si trova un fattore di n;
- se Rabin restituisce $M_i = \left(-C_p M_p - C_q M_q\right) \bmod n$, allora $M - M_i = 2M$ e quindi $\gcd(n, M - M_i) = n$, quindi nemmeno in questo caso si trova un fattore di n;
- se invece Rabin restituisce una delle altre 2 radici, allora si trova un fattore di n

L'algoritmo ha quindi probabilità di errore pari ad $\frac{1}{2}$, dato che restituisce la risposta corretta in 2 casi su 4.

8. Protocollo di ElGamal

Il protocollo di ElGamal è molto simile a quello di Diffie-Hellman per lo scambio di chiavi. Anch'esso si basa sulla difficoltà nel risolvere il **logaritmo discreto**.

Il protocollo di ElGamal era utilizzato soprattutto in ambito open source, in quanto RSA ai tempi era brevettato e c'era da pagare una licenza.

Generazione della chiave (Alice)	**Cifratura** (Bob $\to$ Alice)	**Decifratura** (Alice)
• numero primo p • generatore g del gruppo $\mathbb{Z}_p^*$ • elemento $a \in \mathbb{Z}_p^*$ (chiave privata) • calcola $A = g^a \bmod p$ La chiave pubblica di Alice è la tupla (p, g, A)	1. sceglie un $b \in \mathbb{Z}_p^*$ 2. calcola $B = g^b \bmod p$ La coppia $\left(A^b \cdot M, B\right)$ è il messaggio cifrato.	1. calcola $B^a = g^{ab} = A^b$ 2. calcola $(B^a)^{-1}$ modulo p $M = \cancel{A^b} \cdot \cancel{(B^a)^{-1}} \cdot M$

L'unica differenza rispetto al protocollo di Diffie-Hellman è che il segreto condiviso g^{ab} viene utilizzato per manipolare il messaggio in chiaro.

8. 1. Riuso di b

Il parametro b, con cui si cifra il messaggio da mandare all'altra parte, dev'essere **sempre diverso**. In caso di riuso di b:

$$M_1 = \left(B, A^b M_1\right)$$
$$M_2 = \left(B, A^b M_2\right)$$

Se l'attaccante in un qualche modo è in grado di mettere in chiaro $C_1 = A^b M_1$ (e dunque è in grado di risalire ad A^b invertendo l'equazione) allora è in grado di mettere in chiaro **tutti** i messaggi cifrati con la stessa chiave, perché:

$$\left(A^b\right)^{-1} C_i = M_i$$

(dove C_i è l'i-esimo ciphertext ed M_i è l'i-esimo messaggio cifrato con la stessa chiave b).

8. 2. Sicurezza - ElGamal vs RSA

Il protocollo di ElGamal è considerato **più sicuro** rispetto ad RSA: per ottenere lo stesso livello di sicurezza sono sufficienti chiavi di dimensione **minore**, perché il logaritmo discreto è un problema più difficile della fattorizzazione.

9. Firma digitale

La firma digitale è il risultato più importante raggiunto dalla crittografia asimmetrica, perché aggiunge **autenticazione** alla comunicazione, elemento fondamentale per tutte le comunicazioni sicure.

La firma digitale prevede l'uso delle due chiavi in **ordine inverso** rispetto alla cifratura:

	cifratura	**firma**
cifratura (firma)	Alice usa la chiave **pubblica** di Bob per cifrare il messaggio	Alice usa la sua chiave **privata** per firmare il messaggio
decifratura (verifica della firma)	Bob usa la sua chiave **privata** per decifrare il messaggio di Alice	Bob usa la chiave **pubblica** di Alice per validare la firma sul messaggio che ha ricevuto

9. 1. Uso delle funzioni hash durante la firma digitale

Con la firma digitale si vuole poter essere liberi di firmare messaggi di **dimensione arbitraria**, anche file da diversi MB/GB. Dato il costo computazionale della crittografia asimmetrica, questo sarebbe molto difficile da ottenere se si firmasse il messaggio effettivo. Per questo in realtà la firma non viene fatta sul messaggio, ma sul suo **hash**.

Una volta scelta la funzione di hashing (che dev'essere **crittograficamente sicura**), la dimensione dell'hash rimane costante indipendentemente dalla dimensione del messaggio.

9. 2. Firma digitale con RSA

Per utilizzare RSA come protocollo di firma digitale non c'è bisogno di fare alcuna modifica a quanto già visto per il processo di cifratura, se non usare le due chiavi in ordine inverso.

Dal punto di vista matematico, poiché $M = M^{ed} \bmod n$, le due chiavi e e d sono **simmetriche** (usare e per cifrare e d per decifrare o viceversa porta allo stesso risultato).

9. 2. 1. Blinding attack

Eve vuole trovare un sistema per far firmare ad Alice un messaggio che non firmerebbe mai. L'idea è quella di nascondere il messaggio "malevolo" all'interno di un messaggio "innocuo".

Eve cerca un numero R tale che $\overline{M} = (R^e \cdot M) \bmod n$, dove M è il messaggio "malevolo" ed $\overline{M}$ è il messaggio "innocuo".

Se Alice viene convinta in un qualche modo a firmare $\overline{M}$, si ha:

$$\begin{aligned} F &= \overline{M}^d \bmod n \\ &= [(R^e \cdot M) \bmod n]^d \bmod n \\ &= R^{ed} \cdot M^d \bmod n \\ &= R \cdot M^d \bmod n \end{aligned}$$

Per ottenere il messaggio "malevolo" M firmato da Alice, Eve deve semplicemente calcolare $F \cdot R^{-1} \bmod n = \cancel{R} \cdot \cancel{R^{-1}} \cdot M^d \bmod n$.

Eve non ha bisogno di rubare la chiave privata di Alice, deve soltanto convircela a firmare $\overline{M}$.

Questo attacco è praticabile solo quando il protocollo di firma digitale non prevede l'uso di una funzione di hashing.

9. 3. Firma digitale con ElGamal

A differenza di RSA, il protocollo per la firma non è identico a quello per la cifratura.

Per firmare un messaggio, Alice:
1. sceglie un numero k tale che $\gcd(k, p-1) = 1$;
2. calcola $r = g^k \bmod p$ ed $s = k^{-1}(M - ar) \bmod (p-1)$;
3. invia a Bob $(M, (r, s))$

9. 3. 1. Verifica della firma

Sapendo che $s = k^{-1}(M - ar) \bmod (p-1)$, l'idea potrebbe essere quella di risolvere quest'equazione per M:

$$M = (ks + ar) \bmod (p-1)$$

In realtà però quest'equazione Bob non la può risolvere, perché non conosce nè a (chiave **privata** di Alice) nè k (chiave effimera). Il fatto che Bob (e, in generale, qualunque eavesdropper) non conosca k è essenziale, perché grazie ad esso può risalire alla chiave privata di Alice risolvendo l'equazione per a.

Il fatto che un protocollo di questo tipo sia sbagliato lo si evince anche perché, risolvendo semplicemente l'equazione, non verrebbe mai usata la chiave **pubblica** di Alice.

Dato che $r = g^k \bmod p$ ed s contiene l'inverso di k nella sua definizione, sebbene modulo $(p-1)$ e non modulo p, si calcola $r^s \bmod p$:

$$\begin{aligned} r^s \bmod p &= r^{k^{-1}(M-ar) \bmod (p-1)} \bmod p \\ &= g^{k[k^{-1}(M-ar) \bmod (p-1)]} \bmod p \end{aligned}$$

L'obiettivo ora è togliere il $\bmod (p-1)$ ad esponente. Per farlo bisogna sfruttare un paio di definizioni al fine di riscrivere l'espressione:
1. definizione di inverso modulare: $k^{-1} \bmod (p-1) = t$, con $0 \leq t < p-1$, ovvero $kt = v(p-1) + 1$ per un qualche intero v;
2. definizione di resto: per qualsiasi coppia di interi a e b, $a \bmod b = a - qb$, dove q è il quoziente della divisione di a per b

Mettendo insieme queste due espressioni si può riscrivere l'esponente in questo modo:

$$k^{-1}(M - ar) \bmod (p-1) = t(M - ar) - q(p-1)$$

Inserendo quest'espressione nello sviluppo di $r^s \bmod p$, si ottiene questo:

$$\begin{aligned} r^s \bmod p &= g^{k[k^{-1}(M-ar) \bmod (p-1)]} \bmod p \\ &= g^{k[t(M-ar)-q(p-1)]} \bmod p \\ &= \left(g^{kt(M-ar)} \cdot g^{-kq(p-1)}\right) \bmod p \\ &= \left(g^{[1+v(p-1)](M-ar)} \cdot g^{-kq(p-1)}\right) \bmod p \\ &= (g^{M-ar} \bmod p) \cdot \cancel{\left[(g^{p-1})^{v(M-ar)} \bmod p\right]} \cdot \cancel{\left[(g^{p-1})^{-kq} \bmod p\right]} \\ &= g^{M-ar} \bmod p \\ &= g^M \cdot (g^a \bmod p)^{-r} \bmod p \end{aligned}$$

Nell'ultimo passo è stata messa in evidenza la chiave pubblica di Alice $g^a \bmod p$.

Notare che se l'ultima equazione viene moltiplicata per la chiave pubblica di Alice, ciò che rimane è $g^M \bmod p$.

Per verificare la firma di Alice quindi, Bob:
1. calcola $x_1 = (r^s \cdot g^{ar} \bmod p)$ ed $x_2 = g^M \bmod p$
2. accetta il messaggio solo se $x_1 = x_2$

Per calcolare x_1 Bob ha bisogno della coppia (r, s) ricevuta da Alice, mentre x_2 Bob lo può calcolare autonomamente (visto che il messaggio in chiaro è noto). In sostanza quello che viene fatto è calcolare $g^M \bmod p$ in due modi diversi e accettare il messaggio solo se entrambi questi calcoli danno lo stesso risultato.

9. 3. 2. Riuso di k

Nella firma digitale con ElGamal, k funge da chiave di sessione, dunque è importante non riutilizzarla mai.

Se Alice usa più volte lo stesso k (che implica il riuso dello stesso r), Eve può risalire alla sua chiave **privata** utilizzando soltanto 2 messaggi, perché:

$$\begin{cases} ar + ks_1 = M_1 \bmod p \\ ar + ks_2 = M_2 \bmod p \end{cases}$$

9. 3. 3. Message forgery attack

Il protocollo di ElGamal, se viene firmato il messaggio in sè e non il suo **hash**, ha una vulnerabilità che consente ad Eve di generare **messaggi arbitrari** firmati da Alice, senza bisogno di conoscere la sua chiave privata.

Eve sceglie due numeri x, y tali che $\gcd(y, p-1) = 1$, dopodiché calcola r ed s utilizzando la chiave pubblica di Alice:

$$\begin{aligned} r &= g^x \cdot g^{ab} \bmod p = g^{x+ay} \bmod p \\ s &= -r \cdot y^{-1} \bmod (p-1) \end{aligned}$$

Sapendo che Bob, per verificare il messaggio, controllerà che $r^s \cdot g^{ar} = g^M \bmod p$, Eve pone $M = (xs) \bmod (p-1)$. In questo caso infatti la firma verrà considerata valida da Bob, perché:

$$\begin{aligned} r^s \cdot g^{ar} \bmod p &= g^{ar} \cdot g^{(x+ay)s} \bmod p \\ &= g^{ar+xs+ays} \bmod p \\ &= g^{xs} \bmod p \\ &= g^{(xs) \bmod (p-1)} \bmod p \\ &= g^M \bmod p \end{aligned}$$

Per proteggersi da questo attacco, anziché calcolare l'hash di M si calcola l'hash di $M \parallel r$. Per fare un message forgery attack, Eve a questo punto deve trovare un messaggio $M' \neq M$ tale per cui $H(M' \parallel r) = (xs) \bmod (p-1)$, cosa computazionalmente impossibile se H è first preimage resistant.

9. 4. Firma digitale con DSA

Una particolarità che contraddistingue DSA (Digital Signature Algorithm) dagli altri protocolli di firma digitale è che DSA **fin da subito** prevede l'uso di una funzione di hashing (inizialmente era SHA-1).

9. 4. 1. Generazione delle chiavi

Alice sceglie:
1. un numero primo q di dimensioni pari a quelle dell'output della funzione di hashing (160 bit per SHA-1);

2. un numero primo p da 1024 bit (dimensione gestibile in base al grado di sicurezza che si vuole avere) tale che $p - 1 \bmod q = 0$ (cioè $p - 1$ è un divisore di q) e $\gcd(p - 1, q) = q$.
3. un generatore g del sottogruppo di ordine q di $\mathbb{Z}_p^*$
   - dal **teorema fondamentale dei gruppi ciclici**, poiché p è primo, questo sottogruppo di ordine q di $\mathbb{Z}_p^*$ è **unico**
4. un numero $a \in \mathbb{Z}_p^*$, che sarà la sua chiave **privata**

Dopodiché calcola $A = g^a \bmod p$ ed espone la tupla (p, q, g, A) come chiave **pubblica**.

9. 4. 2. Firma

Dato il messaggio M, Alice:
1. calcola $m = \text{SHA-1}(M)$;
2. sceglie a caso un valore $k \in \mathbb{Z}_q^*$;
3. calcola $r = \left(g^k \bmod p\right) \bmod q$ e $s = k^{-1}(m + ar) \bmod q$
   - se r o s è pari a 0, sceglie un altro k e riprova a generare r ed s
4. invia a Bob la tupla $(M, (r, s))$

9. 4. 3. Verifica della firma

Data la coppia messaggio + firma $(M, (r, s))$ ricevuta da Alice, Bob:
1. recupera la tupla (p, q, g, A) (chiave pubblica di Alice);
2. controlla che $0 < r$ ed $s < q$
   - in caso contrario, scarta immediatamente il messaggio in quanto la firma non è valida
3. calcola $m = \text{SHA-1}(M)$ ed $x = m(s^{-1} \bmod q)$;
4. calcola $y = r(s^{-1} \bmod q)$
5. la firma è valida solo se $(g^x A^y \bmod p) \bmod q = r$

9. 4. 3. 1. Dimostrazione della correttezza

Come nella dimostrazione di ElGamal, ci sarà bisogno di utilizzare le definizioni di inverso modulare e di modulo.

Siano:
- $R = (m + ar)^{-1} \bmod q$;
- Q il quoziente della divisione di kR per q, ovvero il valore tale che $kR \bmod q = kR - Qq$;
- n l'intero tale che $(m + ar)R = 1 + nq$

La catena di uguaglianze è la seguente:

$$
\begin{aligned}
g^x A^y \bmod p &= \left(g^{m(s^{-1} \bmod q)} \cdot g^{ay}\right) \bmod p \\
&= \left(g^{m(s^{-1} \bmod q)} \cdot g^{ar(s^{-1} \bmod q)}\right) \bmod p \\
&= g^{(m+ar)(s^{-1} \bmod q)} \bmod p \\
&= g^{(m+ar)\left[k(m+\text{ a r})^{-1}\right] \bmod q} \bmod p \\
&= g^{(m+ar)(kR \bmod q)} \bmod p \\
&= g^{(m+ar)(kR - Qq)} \bmod p \\
&= g^{kR(m+ar) - Qq(m+ar)} \bmod p \\
&= g^{k(1+nq) - nq(m+ar)} \bmod p \\
&= g^{k + knq - (m+ar)nq} \bmod p \\
&= \left[g^k \cdot \cancel{(g^q)^{nk}} \cdot \cancel{(g^q)^{-(m+ar)n}}\right] \bmod p \\
&= g^k \bmod p
\end{aligned}
$$

Le cancellazioni all'ultimo passaggio sono possibili perché g è un generatore del sottogruppo di ordine q di $\mathbb{Z}_p^*$. Dalla definizione di generatore, moltiplicandolo per sè stesso q volte (pari all'ordine del sottogruppo che genera) si ottiene l'elemento neutro della moltiplicazione, cioè 1.

Dunque, dato che $g^x A^y \bmod p = g^k \bmod p$, per assicurarsi che la firma sia valida Bob deve verificare la seguente uguaglianza:

$$(g^x A^y \bmod p) \bmod q = \underbrace{\left(g^k \bmod p\right) \bmod q}_{r}$$

9. 4. 4. Generazione di p

Per generare un numero primo p dale che $\gcd(p-1, q) = q$, un metodo abbastanza spartano ma efficacie è il seguente:

1. si genera un primo q da 160 bit;
2. si genera un primo r di $1024 - 160 - 1$ bit (dove 1024 è la dimensione in bit che si vuole abbia p);
3. si calcola $p = 2rq + 1$;
4. si controlla se p è primo, e in caso contrario si ripete la procedura

9. 4. 5. Ottenere g

Se g è un generatore del sottogruppo di ordine q di $\mathbb{Z}_p^*$, deve risultare che $g^q \bmod p = 1$.

Sapendo questo, un modo per ricavare g è il seguente:

1. si pone $e = \frac{p-1}{q}$;
2. si sceglie un h a caso tale che $1 < h < p - 1$
   - 1 e $p - 1$ sono esclusi perché andrebbero a generare il sottogruppo di ordine 2 di $\mathbb{Z}_p^*$, dunque sono irrilevanti
3. si calcola $g = h^e \bmod p$
   - se risulta che $g = 1$, si sceglie un nuovo valore h e si ripete la procedura;
   - altrimenti g è il generatore che cerchiamo, infatti:

$$\begin{aligned} g^q \bmod p &= (h^e \bmod p)^q \bmod p \\ &= h^{=} \bmod p \\ &= h^{p\text{-}1} \bmod p \Leftarrow \text{FLT} \\ &= 1 \end{aligned}$$

9. 4. 6. Sicurezza

DSA, come ElGamal e Diffie-Hellman, si basa sulla difficoltà nel calcolare il **logaritmo discreto**. Per q di dimensione 160 bit (quindi usando SHA-1 come funzione di hashing), il livello di sicurezza è $\log\left(\sqrt{2^{160}}\right) = 80$ bit.

9. 5. Autenticità delle chiavi pubbliche

Affinché tutto il sistema di firma digitale (e crittografia asimmetrica in generale) sia robusto, occorre stabilire dei criteri per determinare se una chiave pubblica è **autentica**, ovvero proviene *realmente* da colui che l'ha pubblicata e non da qualcuno che si spaccia per qualcun altro.

Nel corso del tempo si sono sviluppati due approcci: uno **centralizzato** ed uno **decentralizzato**.

9. 5. 1. Approccio TLS

Il protocollo TLS, per garantire l'autenticità delle chiavi pubbliche, si basa sulle **certification authority**. Quando un client si collega ad un server tramite protocollo TLS, il server è tenuto a presentare al client il proprio **certificato**, che è sostanzialmente una chiave pubblica con l'aggiunta di vari metadati (es. periodo di validità, dominio, ecc.).

Il certificato che il server fornisce al client include al suo interno un altro certificato, ovvero quello della **certification authority** che ha **firmato** il certificato del server. Il problema del client non è più quello di verificare l'autenticità della chiave pubblica del server, ma l'autenticità della firma della CA che ha firmato la chiave pubblica del server.

La firma della certification authority può essere a sua volta firmata da un'altra CA che sta più in alto nella gerarchia.

Alla fine, il server non manda al client solo un certificato, ma tutta la **catena** di certificati che va fino alla **root certification authority**.

I certificati delle root certification authority sono installati direttamente nel sistema operativo (e nei browser).

9. 5. 2. Approccio OpenPGP/GPG

L'approccio di OpenPGP non è gerarchico (verticale), ma **peer-to-peer** (orizzontale).

Chiunque può creare una chiave pubblica ed esporla su un **keyserver**.

Ogni chiave pubblica può essere firmata da altre persone. In pratica, più persone firmano una chiave pubblica e più questa è ritenuta affidabile.

OpenPGP mette a disposizione vari strumenti a seconda del grado di fiducia che si vuole dare ad una chiave quando la si firma.

10. Crittografia su curve ellittiche

10. 1. Campi finiti

Un **campo** è un insieme di numeri su cui sono definite le due operazioni di somma e prodotto (ciascuna con il proprio elemento neutro ed inverso). Queste operazioni sono **chiuse** rispetto al campo. La particolarità che distingue un campo da un **gruppo** è che nei campi vale la proprietà distributiva dell'addizione rispetto alla moltiplicazione.

Un campo è **finito** se è composto da un numero finito di elementi. Per p primo, tutti i gruppi $\mathbb{Z}_p$ sono campi finiti. In generale, campi finiti di n elementi esistono se e solo se $n = p^k$, con p primo e $k \geq 1$.

Nel caso in cui $k > 1$, il campo è formato da tutti i polinomi di grado $< k$ con coefficienti in $\mathbb{Z}_p$. Esempio: il campo $GF(3^2)$ è formato da:

- $0, 1, 2$
- $x, x+1, x+2$
- $2x, 2x+1, 2x+2$

Questo campo è comunque chiuso rispetto a somma e moltiplicazione, perché tutte le operazioni vengono ridotte tramite un **polinomio irriducibile**, cioè un polinomio che non può essere riscritto come prodotto di due polinomi distinti.

Esempio: $(2x+1)(x+1) = 2x^2 + 3x + 1$, che non sarebbe nel campo, ma:

- si applica il modulo 3 a tutti i coefficienti: $(2 \bmod 3)x^2 + (3 \bmod 3)x + (1 \bmod 3) = 2x^2 + 1$;
- si calcola il resto della divisione con il polinomio irriducibile per il polinomio che rimane: $(2x^2 + 1) \bmod (x^2 + 1) = 2$

Quindi $(2x+1)(x+1) = 2$ nel campo finito $GF(3^2)$.

L'**ordine** di un campo finito è il numero dei suoi elementi.

La **caratteristica** di un campo finito è il numero di volte con cui si può incrementare di 1 il risultato (partendo da $1+0$) prima di ottenere 0. Nei campi infiniti (ad esempio $\mathbb{R}$) la caratteristica è 0, perché incrementando di 1 non si ottiene mai 0. Nei campi finiti di ordine p^k, invece, la caratteristica è p.

Un campo si dice **algebricamente chiuso** se ogni polinomio ha uno **zero** all'interno del campo. Nessun campo finito è algebricamente chiuso. Un esempio di campo algebricamente chiuso è $\mathbb{C}$, il campo dei numeri complessi.

10. 2. Definizione di curva ellittica

Una **curva ellittica** è un insieme di punti del piano che soddisfa l'equazione

$$y^2 = x^3 + ax + b$$

con a, b coefficienti all'interno del campo F.

Una curva ellittica è **simmetrica rispetto all'asse x**, perché la y compare soltanto al quadrato all'interno dell'equazione. In altre parole, per ogni coppia di punti $(\overline{x}, \overline{y})$ che soddisfa l'equazione, anche la coppia $(\overline{x}, -\overline{y})$ la soddisfa.

Se $P = (x_p, y_p)$, per questioni di notazione si indica con $-P$ la coppia $(x_p, -y_p)$.

10. 3. Curve ellittiche smooth

Curva ellittica smooth: una curva ellittica si dice **smooth** se non esistono punti in cui le **derivate parziali** si annullano simultaneamente.

Le curve smooth sono importanti in ambito crittografico perché è possibile dimostrare che esiste un'**unica tangente** per ogni loro punto.

$\frac{\partial E}{\partial y} = 0 \longrightarrow 2y = 0$	$\frac{\partial E}{\partial x} = 0 \longrightarrow -3x^2 - a = 0$

Tabella 5: Derivate parziali di $E(x, y) = y^2 - x^3 - ax - b$

Riscrivendo la 1° equazione in termini di x:

$$2y = 0 \Longleftrightarrow x^3 + ax + b = 0$$

e mettendo insieme le due equazioni, i valori di x che rendono la curva non-smooth sono le soluzioni di questo sistema:

$$\begin{cases} x^3 + ax + b = 0 \\ 3x^2 + a = 0 \end{cases}$$

Prima di poter risolvere il sistema è necessario gestire alcuni casi particolari:

- se $x = 0$, la curva è non-smooth se e solo se $a = b = 0$. La curva $y^2 = x^3$ ad esempio è non-smooth, perché in $(0, 0)$ entrambe le derivate parziali si annullano. Se invece $a = 0$ e $b \neq 0$ (o viceversa) la curva è smooth, perché le due equazioni non si annullano mai simultaneamente;
- se $a < 0$ ed $x \neq 0$, la curva **potrebbe** essere non smooth se a è sufficientemente negativo da annullare entrambe le equazioni

Imponendo quindi $x \neq 0$ e risolvendo il sistema, si ottiene:

$$\begin{cases} x = -\frac{3b}{2a} \\ \frac{27b^2+4a^3}{4a^2} = 0 \end{cases}$$

La curva quindi è non-smooth se e solo se $27b^2 + 4a^3 = 0$, ed il punto di singolarità ha coordinate $\left(-\frac{3b}{2a}, 0\right)$.

La quantità $27b^2 + 4a^3$ viene detta anche **discriminante**.

10. 4. Intersezioni con una retta

retta standard $y = mx + q$	**retta orizzontale** $y = q$	**retta verticale** $x = c$
$x^3 - mx^2 + (a - 2mq)x + b - q^2 = 0$	$x^3 + ax + b - q^2 = 0$	$y^2 - c^3 - ac - b = 0$

Tabella 6: Coordinate x dei punti d'intersezione di una retta R con E

Nel primo caso e nel 2° caso si ha un polinomio di grado 3, che ha **uno o tre** zeri reali (se ne ha uno solo, gli altri due sono valori **complessi coniugati**).

Nel caso di retta verticale, invece, si ha un polinomio di grado 2 che può avere zeri entrambi reali o entrambi complessi coniugati.

In realtà la situazione è un po' più complessa, perché la curva E è definita su un **campo finito** F, e non è detto che un polinomio di grado **dispari** abbia almeno una soluzione in F. Se però il polinomio ha almeno 2 zeri in F, allora anche il 3° sta in F.

10. 5. Punto all'infinito

L'unico caso rilevante in ambito crittografico è quello in cui esistono **tre** punti d'intersezione tra la curva e la retta, perché è l'unico caso in cui i punti della curva formano un **gruppo** rispetto all'operazione di **addizione**.

Per gestire le **rette verticali**, che hanno solo due punti d'intersezione con la retta senza che ci sia il terzo, è necessario considerare il **punto all'infinito**, indicato con $\mathcal{O}$. Questo punto si ritiene facente parte di **qualunque** curva ellittica E su F.

Il punto $\mathcal{O}$ viene definito in modo da soddisfare alcune proprietà:

- qualsiasi **retta verticale** interseca E in $\mathcal{O}$ con molteplicità 1. Dato che la retta verticale ha già 2 punti d'intersezione con la curva, ne consegue che $\mathcal{O}$ è il terzo e dunque la logica "se ci sono due punti, c'è anche il terzo" funziona anche nel caso di rette verticali;
- nessuna retta non-verticale interseca $\mathcal{O}$;
- nel punto $\mathcal{O}$ la curva ha una tangente t e si suppone che t abbia un'**unica** intersezione con E proprio nel punto $\mathcal{O}$, con molteplicità 3;
- $\mathcal{O} = -\mathcal{O}$

10. 6. Addizione

Se A e B sono due punti di E, la retta che passa per questi due punti interseca E nel punto $-C$. Il punto somma è C, che è il **simmetrico** del punto d'intersezione $-C$ rispetto all'asse x.

L'operazione di addizione configura i punti della curva come **gruppo abelliano**, infatti tutte le proprietà sono rispettate:

- **chiusura**. Questa proprietà è rispettata anche grazie all'introduzione di $\mathcal{O}$, infatti:
  - se $A \neq \mathcal{O}$ e $B \neq \mathcal{O}$, il punto somma è nuovamente un punto sulla curva (eventualmente si tratta di $\mathcal{O}$ stesso);
  - $\mathcal{O} + \mathcal{O} = \mathcal{O}$, la retta tangente ad E nel punto $\mathcal{O}$ ha intersezione con E proprio nel punto $\mathcal{O}$, con molteplicità 3;
  - $A + \mathcal{O} = A$, perché la retta che passa per A e $\mathcal{O}$ interseca la curva nel punto $-A$, il cui simmetrico è proprio A
- esistenza dell'**elemento neutro**: $A + \mathcal{O} = \mathcal{O} + A = A$;
- esistenza dell'**opposto**: $A + (-A) = \mathcal{O}$;
- **associatività**
- **commutatività**. Questa proprietà è "ereditata" dal fatto che la retta passante per A e B è la stessa indipendentemente dall'ordine con cui si considerano i punti.

10. 6. 1. Calcolo delle coordinate di C

10. 6. 1. 1. Caso $A \neq B$

Se $A \neq B$, i punti di intersezione tra E e la retta passante per $A = (x_a, y_a)$ e $B = (x_b, y_b)$ sono le soluzioni del sistema:

$$\begin{cases} y^2 = x^3 + ax + b \\ y = mx + q \end{cases}$$

con m e q definiti in questo modo:

$$m = \frac{y_b - y_a}{x_b - x_a} \qquad q = y_a - mx_a$$

Questo sistema può essere risolto facilmente per sostituzione, sfruttando la 2° equazione $y = mx + q$:

$$(mx + q)^2 = x^3 + ax + b$$

Due delle soluzioni di quest'equazione sono note: sono x_a ed x_b. Si può quindi riscrivere l'equazione in questo modo per mettere in evidenza l'unica incognita $x_{\tilde{c}}$:

$$(x - x_a)(x - x_b)(x - x_{\tilde{c}}) \longrightarrow x_{\tilde{c}} = m^2 - x_a - x_b$$

Una volta ricavato $x_{\tilde{c}}$, per ricavare $y_{\tilde{c}}$ si sfrutta l'equazione della retta:

$$y_{\tilde{c}} = m x_{\tilde{c}} + q$$

e dunque $C = (x_{\tilde{c}}, -y_{\tilde{c}})$.

10. 6. 1. 2. Caso $A = B$ (retta verticale)

In questo caso occorre calcolare il coefficiente angolare della retta tangente ad E nel punto $A = B$.

Anche se la curva non è definita in modo esplicito, è possibile rappresentarla come **unione** dei grafici di due funzioni distinte:

$$y_1(x) = \sqrt{x^3 + ax + b} \qquad y_2(x) = -\sqrt{x^3 + ax + b}$$

queste due funzioni si ottengono calcolando risolvendo per y l'equazione della curva $y^2 = x^3 + ax + b$.

Dato che si parla di tangente, si deve considerare la **derivata** di queste funzioni:

$$y_1'(x) = \frac{3x^2 + a}{2\sqrt{x^3 + ax + b}} = \frac{3x^2 + a}{2y_1(x)} \qquad y_2'(x) = -\frac{3x^2 + a}{2\sqrt{x^3 + ax + b}} = -\frac{3x^2 + a}{2(-y_2(x))}$$

Se $A = B = (x', y')$ è un punto sulla curva, si distinguono i due casi:

$$\begin{cases} y' = y_1'(x') \longrightarrow m = \frac{3x'^2 + a}{2y'} \text{ if } y' \geq 0 \\ y' = y_2(x') \longrightarrow m = \frac{3x'^2 + a}{2y'} \text{ if } y' < 0 \end{cases}$$

dunque il coefficiente angolare m è uguale in entrambi i casi.

```
EC-SUM(A, B):
if A = O:
    return B
if B = O:
    return A
if B = −A:
    return O
if x_a ≠ x_b:
    m ← (y_b − y_a)/(x_b − x_a)
else:
    m ← (3x_a^2 + a)/(2y_a)
q ← y_a − m x_a
return (x_c = m^2 − x_a − x_b, y_c = −(m x_c + q))
```

Figura 16: Algoritmo per il calcolo delle coordinate del punto $C = A + B$

10. 7. Curve ellittiche su $\mathbb{Z}_p$

Anche per le curve definite su $\mathbb{Z}_p$ vale che se un'equazione cubica a coefficienti in $\mathbb{Z}_p$ ha **due** radici in $\mathbb{Z}_p$, allora anche la terza radice è in $\mathbb{Z}_p$.

Questa proprietà consente quindi di definire l'operazione di **addizione** anche per le curve definite su $\mathbb{Z}_p$ e dunque di considerare i punti di questa curva come appartenenti ad un **gruppo**.

Il **sottogruppo** generato da un punto g di una curva $E_{a,b}(\mathbb{Z}_p)$ è definito come:

$$S_{E_{a,b}(\mathbb{Z}_p)}(g) = \{a \in E_{a,b}(\mathbb{Z}_p) \mid \exists k \geq 0, a = k \cdot g\}$$

L'operazione $k \cdot g$, con k intero e g punto della curva, è detta **moltiplicazione scalare**.

In generale, il gruppo definito dai punti di una curva ellittica $E_{a,b}(\mathbb{Z}_p)$ non è ciclico. Tuttavia esiste **almeno un sottogruppo ciclico**.

10. 7. 1. Logaritmo discreto su curva ellittica

Se g è un punto sulla curva $E_{a,b}(\mathbb{Z}_p)$ che genera l'intero gruppo, allora per ogni elemento $z \in E_{a,b}(\mathbb{Z}_p)$ esiste un $k \geq 0$ tale che $z = (kg) \bmod p$.

Il minimo valore di k che soddisfa l'uguaglianza è il **logaritmo a base g di z** e si indica con $k = \log_g x \bmod p$.

Dati k e g è facile calcolare $z = k \cdot g \bmod p$, ma dati z e g risalire a $k = \log_g x \bmod p$ è un problema difficile.

L'algoritmo per il calcolo di z è sulla falsa riga di quello per l'esponenziale modulare (**raddoppiamento ricorsivo**): $2g = g + g$, $4g = 2g + 2g$ e così via.

La difficoltà nel calcolo del logaritmo discreto è il motivo dell'interesse per le curve ellittiche in ambito crittografico. Tuttavia una curva ellittica è un oggetto più complesso di un gruppo numerico $\mathbb{Z}_p$ e che, se non aggiustato opportunamente, rischia di rendere **debole** la crittografia.

Uno dei parametri più importanti da considerare è il **numero di punti** della curva, che permette di definire la dimensione del gruppo additivo. Questo valore può essere considerato l'analogo dell'ordine del gruppo di $\mathbb{Z}_p$ (ed è dunque facile capire se il gruppo è "buono" oppure no), ma il grosso problema è che se calcolare l'ordine di $\mathbb{Z}_p$ è facile, calcolare il numero di punti della curva è decisamente più complesso (non sono noti algoritmi di costo polinomiale).

10. 7. 2. Numero di punti su una curva

Dato il modulo p, il numero di punti di $E_{a,b}(\mathbb{Z}_p)$ è sicuramente $\leq 2p + 1$ (considerando anche il punto $\mathcal{O}$).

Data l'equazione $x^3 + ax + b \bmod p \neq 0$, per ogni valore $x \in \mathbb{Z}_p$, il risultato dell'equazione o è un **residuo quadratico** modulo p oppure non lo è. È ragionevole supporre che il valore sia un residuo quadratico per circa la metà delle $x \in \{0, 1, ..., p - 1\}$.

Se il valore è un residuo quadratico, allora ha **due** radici in $\mathbb{Z}_p$ ed ogni radice identifica un punto sulla curva. Quindi:

- se il valore è un residuo quadratico, vengono identificati 2 punti sulla curva;
- se il valore non è un residuo quadratico, non viene identificato nessun punto

da cui segue che il numero di punti debba essere all'incirca dell'ordine di p.

Teorema di Hasse: il massimo numero di punti di una curva $E_{a,b}(\mathbb{Z}_p)$ è $\leq 2\sqrt{p}$.

L'algoritmo più efficiente per il calcolo del numero di punti di una curva è quello di **René Schoof**, che ha costo $O\left(\log(p)^8\right)$. Il costo è quindi polinomiale, ma l'esponente è elevato, dunque

non è esattamente un algoritmo efficiente. C'è di buono però che una volta stabiliti i coefficienti a, b da cui dipende la curva, il calcolo del numero di punti dev'essere fatto **una sola volta**.

Nella pratica, le curve ellittiche sicure per contesti crittografici sono ben note (la lista è publicamente disponibile sul web), dunque non c'è mai bisogno di calcolare questo valore perché è già stato calcolato da qualcun altro.

10. 7. 3. Attacco al logaritmo discreto su curve ellittiche

Le informazioni pubbliche in un protocollo basato su curva ellittica sono:

- l'equazione della curva E, cioè il modulo p ed i due parametri a, b da cui la curva dipende;
- un punto P che è il generatore di un **sottogruppo ciclico** formato da tutti i punti della curva;
- un secondo punto $Q = k \cdot P$, per un qualche valore **segreto** k

L'attaccante vuole risalire al valore di k. Per farlo deve trovare due coppie di valori $(\alpha_P, \alpha_Q), (\beta_P, \beta_Q)$ tali da verificare la combinazione lineare $\alpha_P P + \alpha_Q Q = \beta_P P + \beta_Q Q$, che sostituendo $Q = k \cdot P$ diventa:

$$(\alpha_P + k\alpha_Q)P = (\beta_P + k\beta_Q)P$$

I due coefficienti $(\alpha_P + k\alpha_Q)$ e $(\beta_P + k\beta_Q)$ devono esseere **congrumenti modulo N**, dove N è il numero di punti di E, perché, per il teorema di Lagrange, l'ordine del sottogruppo generato da P è un divisore dell'ordine del gruppo dei punti sulla curva $\#E$, quindi se $\alpha_P + \alpha k_Q$ e $\beta_P + k\beta_Q$ sono congrui modulo $\#E$, sono anche congrui modulo l'ordine del sottogruppo.

In altre parole, se esistono coefficienti che rendono vera $\alpha_P - \beta_P \equiv k(\beta_Q - \alpha_Q) \bmod \#E$, allora se $\gcd(\beta_Q - \alpha_Q, \#E) = 1$ esiste l'inverso di $\beta_Q - \alpha_Q \bmod \#E$ e dunque si può risalire al valore di k:

$$k = (\alpha_P - \beta_P)(\beta_Q - \alpha_Q)^{-1} \bmod \#E$$

10. 7. 3. 1. Complessità dell'algoritmo

Con una dimostrazione simile a quella del **paradosso del compleanno**, si può dire che per trovare questa coppia di coefficienti occorrono circa $\sqrt{2^n}$ operazioni per numeri da n bit.

10. 8. Diffie-Hellman su curve ellittiche (ECDH)

1. Alice e Bob concordano una curva ellittica E da utilizzare e su un punto base $P \in E$;
2. Alice sceglie a caso un valore k_a (chiave privata) ed invia $A = k_a \cdot P$ a Bob (chiave pubblica);
3. Bob sceglie a caso un valore k_b (chiave privata) ed invia $B = k_b \cdot P$ ad Alice (chiave pubblica);
4. Alice calcola $Z_A = k_a \cdot B = k_a \cdot k_b \cdot P$;
5. Bob calcola $Z_B = k_b \cdot A = k_b \cdot k_a \cdot P$;
6. il valore $Z = Z_A = Z_B$ è il segreto condiviso
7. entrambi applicano una **funzione di hashing** per calcolare la **chiave simmetrica** $h = H(Z)$ che verrà usata nelle successive comunicazioni tra Alice e Bob

Il costo complessivo per il calcolo dei prodotti scalari $k_a \cdot P$, $k_b \cdot P$, $k_a \cdot B$ e $k_b \cdot A$ è di $O(n^3)$ (con n numero di bit delle chiavi segrete).

10. 8. 1. Attacco basato sull'uso di una curva debole

L'attacco parte dalla seguente considerazione: dalle formule per il calcolo di $C = A + B$, che passano per il calcolo di m, non compare mai il parametro b della curva ellittica, mentre il parametro a compare soltanto nel caso in cui $A = B$:

$$\begin{cases} m = \frac{y_b - y_a}{x_b - x_a} \bmod p \text{ if } A \neq B \\ m = \frac{3x_a^2 + a}{2y_a} \bmod p \text{ if } A = B \end{cases}$$

Sebbene possa sembrare strano, in entrambi i casi la curva ellittica è sempre ben definita:

- se $A \neq B$ esiste una sola curva che passa per questi 2 punti;
- se $A = B$ ma il parametro a è definito, esiste una sola curva con quel valore a e che passa per il punto $A = B$

Se Bob non controlla che il punto somma C ricevuto da Alice sia interno alla curva, Alice può utilizzare per suo conto una **curva più debole** rispetto a quella concordata con Bob al fine di risalire alla sua chiave privata k_b.

10. 8. 1. 1. Esempio

1. Alice e Bob concordano sull'uso di $E_{-4,0}(\mathbb{Z}_{10531})$ e $P = (339, 115)$.

I punti di questa curva non formano un gruppo ciclico, perché:

- il numero di punti della curva in questo caso è esattamente $p + 1 = 10532$, che ha fattorizzazione $2^2 \cdot 2633$;
- se si prendono tutti i punti della curva e si controlla l'ordine del sottogruppo generato da ogni punto, si hanno soltanto 3 sottogruppi di ordine rispettivamente $\{2633, 2, 5266\}$. Mancano il sottogruppo di ordine 4 e quello di ordine pari alla dimensione dell'intero gruppo.

Il punto $P = (339, 115)$ è però una buona scelta, perché, tra questi 3 sottogruppi, genera quello col più alto numero di elementi.

2. Bob sceglie la sua chiave segreta k_b ed invia ad Alice $B = k_b \cdot P$.
3. Alice, per calcolare il suo A, altera il parametro b della curva E, poiché, come già visto, il calcolo del punto somma non dipende da questo parametro. Sceglie quindi di usare la curva $E_{-4,1}(\mathbb{Z}_{10531})$ e un punto $A = (9123, 1166)$.

Il numero di elementi della curva di Alice è 10481, che ha fattorizzazione $47 \cdot 223$, ed il punto A scelto da Alice è generatore del sottogruppo di ordine 47.

Questo sottogruppo è di ordine (estremamente) più basso rispetto a quello del sottogruppo generato dal punto P usato da Bob (che ha ordine 5266).

4. Bob **non controlla** che l'A ricevuto da Alice stia sulla curva $E_{4,0}(\mathbb{Z}_{10531})$, ma calcola il segreto condiviso $Z = k_b \cdot A$

Poiché l'ordine di A è 47 e Z è un multiplo di A, ne segue che l'ordine del sottogruppo generato da Z è a sua volta 47.

Se Q ed R sono rispettivamente quoziente e resto della divisione intera per 47, allora $k_b = Q \cdot 47 + R$, ovvero $Z = k_b \cdot A = R \cdot A$. In altre parole, per trovare k_b è sufficiente trovare il **resto** della sua divisione per 47, e questo può essere fatto anche tramite **bruteforce** perché R deve necessariamente essere un valore $0 \leq R < 47$.

Il valore Z calcolato da Bob quindi non è un punto sulla curva $E_{-4,0}(\mathbb{Z}_{10531})$, ma è un punto sulla curva debole di Alice $E_{-4,1}(\mathbb{Z}_{10531})$.

5. Bob, ignaro di tutto questo, calcola $h = H(Z)$ ed invia un messaggio ad Alice cifrato con la chiave h;

6. per decifrare il messaggio, Alice deve trovare tramite bruteforce un valore k_{b_1} nel range intero $[0, 47)$ tale che il messaggio decifrato con la chiave $h' = H\left(Z' = k_{b_1} \cdot A\right)$ non sia pattume.
7. a questo punto Alice è riuscita a decifrare il messaggio, ma non ha (ancora) modo di conoscere la chiave privata k_b di Bob. Per ora ha soltanto trovato il valore k_{b_1} tale che $k_b \bmod 47 = k_{b_1}$.

Tuttavia Alice può **ripetere il giro** convincendo Bob ad utilizzare un altro punto A' ed a mandargli un nuovo messaggio cifrato.

Tramite bruteforce Alice troverà un altro valore k_{b_2} tale che $k_b \bmod N = k_{b_2}$, dove N è l'ordine del sottogruppo generato dal punto A' (valore che è noto ad Alice).

Per risalire a k_b ora Alice deve semplicemente usare il CRT per risolvere il sistema:

$$\begin{cases} k_b \bmod 47 = k_{b_1} \\ k_b \bmod N = k_{b_2} \end{cases}$$

dove l'unica incognita è k_b.

10. 8. 1. 2. Considerazioni

Questo attacco è percorribile solo se Bob sta mandando dei **messaggi di testo**, altrimenti Alice non saprebbe quando fermarsi nel bruteforce. Tuttavia questo è un problema poco rilevante per l'attaccante, perché a questo punto della comunicazione (con il segreto condiviso già scambiato tra Alice e Bob) è **molto probabile** che i messaggi siano testuali.

Per difendersi da questo attacco Bob, dopo aver calcolato Z, deve controllare che Z stia sulla curva che sta pensando di utilizzare. Sebbene possa sembrare scontato, nelle prime implementazioni della crittografia su curve ellittiche questo controllo non veniva fatto, rendendo possibile l'attacco descritto sul protocollo TLS.

10. 9. DSA su curve ellittiche (ECDSA)

Per generare la chiave, Alice:
1. sceglie una curva ellittica $E_{a,b}(\mathbb{Z}_p)$ e un punto base $P \in E_{a,b}(\mathbb{Z}_p)$;
2. determina il **numero di punti** della curva $N = \#E_{a,b}(\mathbb{Z}_p)$;
3. sceglie a caso un valore $a \in \mathbb{Z}_p$ (chiave privata);
4. calcola $A = a \cdot P$ (chiave pubblica)

Per firmare un messaggio, Alice:
1. calcola $h = H(M) \bmod N$ (dove H è una qualche **funzione di hashing**)
   - per costruzione, $0 \leq h < N$
2. sceglie un valore $k \in [0, N)$ tale che $\gcd(k, N) = 1$ e calcola $Q = k \cdot P = (Q_x, Q_y)$;
3. calcola $r = Q_x \bmod N$ ed $s = k^{-1}(h + ra) \bmod N$;
4. invia a Bob la coppia $(M, (r, s))$

Per validare la firma, Bob:
1. calcola $w = s^{-1} \bmod N = k(h + ra)^{-1} \bmod N$;
2. calcola $u = hw$ e $v = rw$, dove h è l'hash del messaggio;
3. determina il punto $R = (uP + vA) \bmod N = (R_x, R_y)$;
4. accetta il messaggio se e solo se $R_x = r = Q_x \bmod N$

10. 9. 1. Dimostrazione della correttezza

$$\begin{aligned} R &= uP + vA \\ &= hwP + rwA \\ &= hwP + rwaP \\ &= w(h + ra)P \\ &= k\cancel{(h + ra)^{-1}}\cancel{(h + ra)}P \\ &= kP \\ &= Q \end{aligned}$$

10. 9. 2. Attacco in caso di riuso di k

Anche la versione su curva ellittica è vulnerabile se k viene riutilizzato.

Riutilizzare k per messaggi diversi porta ad avere anche lo stesso valore di r per entrambi i messaggi.

Dati due hash h_1, h_2 di due messaggi e le due firme $(r, s_1), (r, s_2)$, l'attaccante può calcolare:

$$\begin{aligned} s_1 - s_2 &= (h_1 + ra)k^{-1} - (h_2 + ra)k^{-1} \\ &= (h_1 + ra - h_2 - ra)k^{-1} \\ &= (h_1 - h_2)k^{-1} \end{aligned}$$

da cui può calcolare facilmente il valore di k:

$$k = (h_1 - h_2)(s_1 - s_2)^{-1} \bmod N$$

Ovviamente l'attacco funziona solo se $\gcd(s_1 - s_2, N) = 1$. In generale, se la curva ellittica è scelta bene, è raro che $s_1 - s_2$ sia un divisore di N, ma in ogni caso se anche dovesse capitare l'attaccante può sempre ritentare il calcolo di k al messaggio successivo (che avrà due valori s_1, s_2 differenti).

Una volta trovato k si può risalire alla chiave privata:

$$\begin{aligned} (k \cdot s_1 - h_1)r^{-1} &= [(h_1 + ra) - h_1]r^{-1} \\ &= (r + a)r^{-1} \\ &= a \end{aligned}$$

10. 10. Vantaggi e svantaggi delle curve ellittiche

Il problema principale delle curve ellittiche è che è molto difficile trovarne di **buone** per scopi crittografici.

Il grosso vantaggio delle curve ellittiche è che permettono di usare delle chiavi decisamente più **piccole** rispetto alla crittografia senza curve ellittiche: 256 bit danno già una curva molto sicura, contro i 2.048 bit richiesti per RSA oggi.